

Reinforcement Learning

一、Shiyu Zhao's 【强化学习的数学原理】

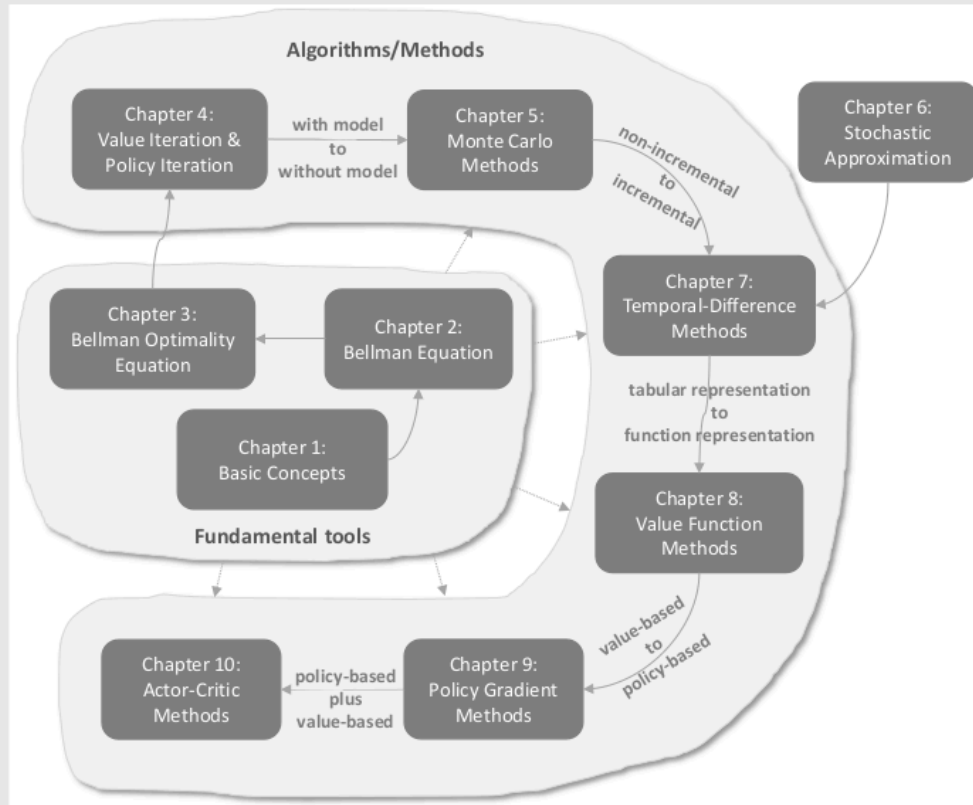
Source:

- [Lectures](#) (bilibili)
- [Slides](#) (github)
- [Note](#) (feishu)

Code block

```
1  Name:Zhongchun Chen
2  email: pocket-chen@sjtu.edu.cn
3
4  about the note:
5  1> Remark often after the content
6  2> code block expresses some transition thinking
7  3> Yellow base highlights some TODO@name, which usually is something
    interesting waited
```

0、Outline



1. Basic Concepts in RL

Markov decision process (MDP)

Key elements of MDP:

- Sets:
 - State: the set of states $\mathcal{S}$
 - Action: the set of actions $\mathcal{A}(s)$ is associated for state $s \in \mathcal{S}$.
 - Reward: the set of rewards $\mathcal{R}(s, a)$.
- Probability distribution:
 - State transition probability: at state s , taking action a , the probability to transit to state s' is $p(s'|s, a)$
 - Reward probability: at state s , taking action a , the probability to get reward r is $p(r|s, a)$
- Policy: at state s , the probability to choose action a is $\pi(a|s)$
- Markov property: memoryless property •

$$p(s_{t+1}|a_{t+1}, s_t, \dots, a_1, s_0) = p(s_{t+1}|a_{t+1}, s_t),$$

$$p(r_{t+1}|a_{t+1}, s_t, \dots, a_1, s_0) = p(r_{t+1}|a_{t+1}, s_t).$$

All the concepts introduced in this lecture can be put in the framework in MDP.

Remark:

- MDP corresponds to (M: Markov memoryless property) + (D: decision policy) + (P: process depend on model or environment)
- In our frame-rules, when arriving at the end object, it will still get a reward uninfinitely, depending on discount rate to less and less

- Not allow it goes out the goal state, not use here
- Allow it goes out, use here, more normally

2、 Bellman Equation

2.0 Summary

Key concepts and results:

- State value: $v_\pi(s) = \mathbb{E}[G_t | S_t = s]$
- Action value: $q_\pi(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$
- The Bellman equation (elementwise form):

$$\begin{aligned} v_\pi(s) &= \sum_a \pi(a|s) \left[\underbrace{\sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_\pi(s')}_{q_\pi(s, a)} \right] \\ &= \sum_a \pi(a|s) q_\pi(s, a) \end{aligned}$$

- The Bellman equation (matrix-vector form):

$$v_\pi = r_\pi + \gamma P_\pi v_\pi$$

- How to solve the Bellman equation: closed-form solution, iterative solution

2.1 deterministic form (matrix P represents policy ,where r_i is determined, not variable exception)

How to solve these equations? Write in the following matrix-vector form:

$$\underbrace{\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix}}_{\mathbf{v}} = \underbrace{\begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ r_4 \end{bmatrix}}_{\mathbf{r}} + \underbrace{\begin{bmatrix} \gamma v_2 \\ \gamma v_3 \\ \gamma v_4 \\ \gamma v_1 \end{bmatrix}}_{\gamma \mathbf{P} \mathbf{v}} = \underbrace{\begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ r_4 \end{bmatrix}}_{\mathbf{r}} + \gamma \underbrace{\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}}_{\mathbf{P}} \underbrace{\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix}}_{\mathbf{v}}$$

which can be rewritten as

$$\mathbf{v} = \mathbf{r} + \gamma \mathbf{P} \mathbf{v}$$

This is the Bellman equation (for this specific deterministic problem)!!

- Though simple, it demonstrates the core idea the value of one state relies on the values of other states.
- A matrix-vector form is more clear to see how to solve the state values.

2.2 state transition represented by space(probability ditrubution)

Random variable $X \rightarrow$ usually think about Expectation $E[X]$

Consider the following single-step process:

$$S_t \xrightarrow{A_t} R_{t+1}, S_{t+1}$$

- $t, t+1$: discrete time instances
- S_t : state at time t
- A_t : the action taken at state S_t
- R_{t+1} : the reward obtained after taking A_t
- S_{t+1} : the state transited to after taking A_t

Note that S_t, A_t, R_{t+1} are all *random variables*.

This step is governed by the following probability distributions:

- $S_t \rightarrow A_t$ is governed by $\pi(A_t = a | S_t = s)$
- $S_t, A_t \rightarrow R_{t+1}$ is governed by $p(R_{t+1} = r | S_t = s, A_t = a)$
- $S_t, A_t \rightarrow S_{t+1}$ is governed by $p(S_{t+1} = s' | S_t = s, A_t = a)$

At this moment, we assume we know the model (i.e., the probability distributions)!

Remark:

- Usually call distributions $P(R_{t+1} | S_t, A_t)$ and $P(S_{t+1} | S_t, A_t)$ as environment or model (model-based vs model-free)
- Call distributions $\pi(A_t | S_t)$ as policy

2.3 Return G also random variable

Consider the following multi-step trajectory:

$$S_t \xrightarrow{A_t} R_{t+1}, S_{t+1} \xrightarrow{A_{t+1}} R_{t+2}, S_{t+2} \xrightarrow{A_{t+2}} R_{t+3}, \dots$$

The discounted return is

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

- $\gamma \in [0, 1)$ is a discount rate.
- G_t is also a random variable since $R_{t+1}, R_{t+2}, \dots$ are random variables.

2.4 State Value (Function) is Expectation of Return $E[G]$

The expectation (or called expected value or mean) of G_t is defined as the *state-value function* or simply *state value*:

$$v_{\pi}(s) = \mathbb{E}[G_t | S_t = s]$$

Remarks:

- It is a function of s . It is a conditional expectation with the condition that the state starts from s .
- It is based on the policy π . For a different policy, the state value may be different.

Q: What is the relationship between return and state value?

A: The state value is the mean of all possible returns that can be obtained starting from a state. If everything - $\pi(a|s)$, $p(r|s, a)$, $p(s'|s, a)$ - is deterministic, then state value is the same as return.

2.5 Bellman Equation Def in elementwise form

The return G_t can be written as

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots, \\ &= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots), \\ &= R_{t+1} + \gamma G_{t+1}, \end{aligned}$$

Then, it follows from the definition of the state value that

$$\begin{aligned} v_{\pi}(s) &= \mathbb{E}[G_t | S_t = s] \\ &= \mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = s] \\ &= \mathbb{E}[R_{t+1} | S_t = s] + \gamma \mathbb{E}[G_{t+1} | S_t = s] \end{aligned}$$

Therefore, we have

$$\begin{aligned} v_{\pi}(s) &= \mathbb{E}[R_{t+1} | S_t = s] + \gamma \mathbb{E}[G_{t+1} | S_t = s], \\ &= \underbrace{\sum_a \pi(a|s) \sum_r p(r|s, a) r}_{\text{mean of immediate rewards}} + \gamma \underbrace{\sum_a \pi(a|s) \sum_{s'} p(s'|s, a) v_{\pi}(s')}_{\text{mean of future rewards}}, \\ &= \sum_a \pi(a|s) \left[\sum_r p(r|s, a) r + \gamma \sum_{s'} p(s'|s, a) v_{\pi}(s') \right], \quad \forall s \in \mathcal{S}. \end{aligned}$$

Highlights:

- The above equation is called the *Bellman equation*, which characterizes the relationship among the state-value functions of different states.
- It consists of two terms: the immediate reward term and the future reward term.
- A set of equations: every state has an equation like this!!!

Remark:

- The equation is elementwise, which means every state has one bellman equation like above (n state - n $v_{\pi}(s)$ - n equations)
- If we combine all elementwise state bellman equations, it becomes one matrix-vector form
- Bellman Equation other equivalent forms

i.
$$v_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r|s, a) [r + \gamma v_{\pi}(s')]$$

ii.
$$v_{\pi}(s) = \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s], s \in \mathcal{S}$$
 called as Bellman expectation equation

iii. If reward r just depends on the next state s' , $r = r(s')$

$$v_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} p(s'|s, a) [r(s') + \gamma v_{\pi}(s')]$$

2.6 Bellman Equation in matrix-vector form

2.6.1 Bellman Equation Def / Expression in matrix-vector form

Suppose the states could be indexed as s_i ($i = 1, \dots, n$).

For state s_i , the Bellman equation is

$$v_\pi(s_i) = r_\pi(s_i) + \gamma \sum_{s_j} p_\pi(s_j|s_i) v_\pi(s_j)$$

Put all these equations for all the states together and rewrite to a matrix-vector form

$$v_\pi = r_\pi + \gamma P_\pi v_\pi$$

where

- $v_\pi = [v_\pi(s_1), \dots, v_\pi(s_n)]^T \in \mathbb{R}^n$
- $r_\pi = [r_\pi(s_1), \dots, r_\pi(s_n)]^T \in \mathbb{R}^n$
- $P_\pi \in \mathbb{R}^{n \times n}$, where $[P_\pi]_{ij} = p_\pi(s_j|s_i)$, is the *state transition matrix*

where

$$r_\pi(s) \triangleq \sum_a \pi(a|s) \sum_r p(r|s, a) r, \quad p_\pi(s'|s) \triangleq \sum_a \pi(a|s) p(s'|s, a)$$

Remark:

- r_π is the expectation of reward in distributions of policy $\pi(a|s)$ and model $p(r|s, a)$
- One example

$$\underbrace{\begin{bmatrix} v_\pi(s_1) \\ v_\pi(s_2) \\ v_\pi(s_3) \\ v_\pi(s_4) \end{bmatrix}}_{v_\pi} = \underbrace{\begin{bmatrix} r_\pi(s_1) \\ r_\pi(s_2) \\ r_\pi(s_3) \\ r_\pi(s_4) \end{bmatrix}}_{r_\pi} + \gamma \underbrace{\begin{bmatrix} p_\pi(s_1|s_1) & p_\pi(s_2|s_1) & p_\pi(s_3|s_1) & p_\pi(s_4|s_1) \\ p_\pi(s_1|s_2) & p_\pi(s_2|s_2) & p_\pi(s_3|s_2) & p_\pi(s_4|s_2) \\ p_\pi(s_1|s_3) & p_\pi(s_2|s_3) & p_\pi(s_3|s_3) & p_\pi(s_4|s_3) \\ p_\pi(s_1|s_4) & p_\pi(s_2|s_4) & p_\pi(s_3|s_4) & p_\pi(s_4|s_4) \end{bmatrix}}_{P_\pi} \underbrace{\begin{bmatrix} v_\pi(s_1) \\ v_\pi(s_2) \\ v_\pi(s_3) \\ v_\pi(s_4) \end{bmatrix}}_{v_\pi}.$$

- Row sum is 1 and all elements > 0

2.6.2 Solution to the Bellman Equation

- The *closed-form solution* is:

$$v_\pi = (I - \gamma P_\pi)^{-1} r_\pi$$

- The matrix $I - \gamma P_\pi$ is invertible. See details in my book.
- We still need to use numerical algorithms to calculate the matrix inverse.
- Can we avoid the matrix inverse operation? Yes, as shown below.

- An *iterative solution* is:

$$v_{k+1} = r_\pi + \gamma P_\pi v_k$$

This algorithm leads to a sequence $\{v_0, v_1, v_2, \dots\}$. We can show that

$$v_k \rightarrow v_\pi = (I - \gamma P_\pi)^{-1} r_\pi, \quad k \rightarrow \infty$$

2.7 Action Value

Definition:

$$q_{\pi}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$$

- $q_{\pi}(s, a)$ is a function of the state-action pair (s, a)
- $q_{\pi}(s, a)$ depends on π

It follows from the properties of conditional expectation that

$$\underbrace{\mathbb{E}[G_t | S_t = s]}_{v_{\pi}(s)} = \sum_a \underbrace{\mathbb{E}[G_t | S_t = s, A_t = a]}_{q_{\pi}(s, a)} \pi(a|s)$$

Hence,

$$v_{\pi}(s) = \sum_a \pi(a|s) q_{\pi}(s, a) \quad (2)$$

Recall that the state value is given by

$$v_{\pi}(s) = \sum_a \pi(a|s) \left[\underbrace{\sum_r p(r|s, a) r + \gamma \sum_{s'} p(s'|s, a) v_{\pi}(s')}_{q_{\pi}(s, a)} \right] \quad (3)$$

By comparing (2) and (3), we have the **action-value function** as

$$q_{\pi}(s, a) = \sum_r p(r|s, a) r + \gamma \sum_{s'} p(s'|s, a) v_{\pi}(s') \quad (4)$$

(2) and (4) are the **two sides of the same coin**:

- (2) shows how to obtain state values from action values.
- (4) shows how to obtain action values from state values.

Remark:

- $q_{\pi}(s, a)$ ~~(a) depends on π just when evaluating state value.~~ Yet during iteration a can be anything to calculate $q_{\pi}(s, a)$ to find better policy

3、 Bellman Optimality Equation

3.1 What is best / better policy

Definition

A policy π^* is optimal if $v_{\pi^*}(s) \geq v_{\pi}(s)$ for all s and for any other policy π .

Remark: best policy π^* whether exist and whether unique

3.2 Bellman Optimality Equation

Bellman optimality equation (elementwise form):

$$\begin{aligned} v(s) &= \max_{\pi} \sum_a \pi(a|s) \left(\sum_r p(r|s, a) r + \gamma \sum_{s'} p(s'|s, a) v(s') \right), \quad s \in \mathcal{S} \\ &= \max_{\pi} \sum_a \pi(a|s) q(s, a), \quad s \in \mathcal{S} \end{aligned}$$

Remarks:

- $p(r|s, a), p(s'|s, a), r, \gamma$ are known.
- $v(s), v(s')$ are unknown and to be calculated.
- Is $\pi(s)$ known or unknown?

Bellman optimality equation (matrix-vector form):

$$v = \max_{\pi} (r_{\pi} + \gamma P_{\pi} v)$$

where the elements corresponding to s or s' are

$$\begin{aligned} [r_{\pi}]_s &\triangleq \sum_a \pi(a|s) \sum_r p(r|s, a) r, \\ [P_{\pi}]_{s, s'} &= p(s'|s) \triangleq \sum_a \pi(a|s) \sum_{s'} p(s'|s, a) \end{aligned}$$

- This lecture will answer all the following questions:
 - Algorithm: how to solve this equation?
 - Existence: does this equation have solutions?
 - Uniqueness: is the solution to this equation unique?
 - Optimality: how is it related to optimal policy?

Question: π is unknown and how to solve the equation?

Solution: Suppose fix or know $v(s'), q(s, a), \pi$ select $\max_a^* = \operatorname{argmax}_a q(s, a)$. In other words, π depends on $v(s'), q(s, a), \pi = f(v(\mathcal{S})) = f(v)$, yet we couldn't find the exited function f

Fix $v'(s)$ first and solve π :

$$\begin{aligned} v(s) &= \max_{\pi} \sum_a \pi(a|s) \left(\sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v(s') \right), \quad \forall s \in \mathcal{S} \\ &= \max_{\pi} \sum_a \pi(a|s) q(s, a) = \max_{\pi} [\pi(a_1|s)q(s, a_1) + \dots + \pi(a_5|s)q(s, a_5)] \\ &\doteq \max_{c_1, \dots, c_5} [c_1 q(s, a_1) + \dots + c_5 q(s, a_5)], \quad c_1 + \dots + c_5 = 1 \end{aligned}$$

Example (How to solve $\max_{\pi} \sum_a \pi(a|s)q(s, a)$)

Suppose $q_1, q_2, q_3 \in \mathbb{R}$ are given. Find c_1^*, c_2^*, c_3^* solving

$$\max_{c_1, c_2, c_3} c_1 q_1 + c_2 q_2 + c_3 q_3.$$

where $c_1 + c_2 + c_3 = 1$ and $c_1, c_2, c_3 \geq 0$.

Answer: Suppose $q_3 \geq q_1, q_2$. Then, the optimal solution is $c_3^* = 1$ and $c_1^* = c_2^* = 0$. That is because for any c_1, c_2, c_3

$$q_3 = (c_1 + c_2 + c_3)q_3 = c_1 q_3 + c_2 q_3 + c_3 q_3 \geq c_1 q_1 + c_2 q_2 + c_3 q_3.$$

Inspired by the above example, considering that $\sum_a \pi(a|s) = 1$, we have

$$\begin{aligned} v(s) &= \max_{\pi} \sum_a \pi(a|s) \left(\sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v(s') \right), \quad \forall s \in \mathcal{S} \\ &= \max_{\pi} \sum_a \pi(a|s) q(s, a) \\ &= \max_{a \in \mathcal{A}(s)} q(s, a) \end{aligned}$$

where the optimality is achieved when

$$\pi(a|s) = \begin{cases} 1 & a = a^* \\ 0 & a \neq a^* \end{cases}$$

where $a^* = \arg \max_a q(s, a)$.

Remark: equation right side only has v can be defined as $f(v)$, which is different from above

The BOE is $v = \max_{\pi} (r_{\pi} + \gamma P_{\pi} v)$. Let

$$f(v) := \max_{\pi} (r_{\pi} + \gamma P_{\pi} v)$$

Then, the Bellman optimality equation becomes

$$v = f(v)$$

where

$$[f(v)]_s = \max_{\pi} \sum_a \pi(a|s) q(s, a), \quad s \in \mathcal{S}$$

Remark: By the Bellman Optimality Equation $v = f(v)$, we can explain **Existence, Uniqueness** and **Algorithm(Iteration!)**.

Math Theory is as follows:

Some concepts:

- **Fixed point:** $x \in X$ is a fixed point of $f : X \rightarrow X$ if

$$f(x) = x$$

- **Contraction mapping (or contractive function):** f is a contraction mapping if

$$\|f(x_1) - f(x_2)\| \leq \gamma \|x_1 - x_2\|$$

where $\gamma \in (0, 1)$.

- γ must be strictly less than 1 so that many limits such as $\gamma^k \rightarrow 0$ as $k \rightarrow \infty$ hold.
- Here $\|\cdot\|$ can be any vector norm.

Theorem (Contraction Mapping Theorem)

For any equation that has the form of $x = f(x)$, if f is a contraction mapping, then

- **Existence:** there exists a fixed point x^* satisfying $f(x^*) = x^*$.
- **Uniqueness:** The fixed point x^* is unique.
- **Algorithm:** Consider a sequence $\{x_k\}$ where $x_{k+1} = f(x_k)$, then $x_k \rightarrow x^*$ as $k \rightarrow \infty$. Moreover, the convergence rate is exponentially fast.

Through it, we can get **iteration-algorithm**

Theorem (Existence, Uniqueness, and Algorithm)

For the BOE $v = f(v) = \max_{\pi}(r_{\pi} + \gamma P_{\pi}v)$, there always **exists** a solution v^* and the solution is **unique**. The solution could be solved iteratively by

$$v_{k+1} = f(v_k) = \max_{\pi}(r_{\pi} + \gamma P_{\pi}v_k) \quad (1)$$

This sequence $\{v_k\}$ converges to v^* **exponentially fast** given any initial guess v_0 . The convergence rate is determined by γ .

Theorem (Policy Optimality)

Suppose that v^* is the unique solution to $v = \max_{\pi}(r_{\pi} + \gamma P_{\pi}v)$, and v_{π} is the state value function satisfying $v_{\pi} = r_{\pi} + \gamma P_{\pi}v_{\pi}$ for any given policy π , then

$$v^* \geq v_{\pi}, \quad \forall \pi$$

Remark: in the above communication, policy update is the **deterministic Greedy Optimal Policy**

Theorem (Greedy Optimal Policy)

For any $s \in S$, the deterministic greedy policy

$$\pi^*(a|s) = \begin{cases} 1 & a = a^*(s) \\ 0 & a \neq a^*(s) \end{cases}$$

is an optimal policy solving the BOE. Here,

$$a^*(s) = \arg \max_a q^*(a, s),$$

3.3 Analyze parameters' effects on optimal policies

- Reward r
 - Reward r matters are their relative values, not their absolute values.
 - Give unwanted action less r , which will punish this action more seriously. (set in real situation and by your idea)
- Discount rate $\gamma \in (0, 1)$

- Larger γ : the policy thinks more about the future
- Little γ : the policy thinks more about current

4、 Iteration Algorithm

Code block

```

1  Through above communication, we noted that iteration can be find the solution
   of
2  the bellman equation "v=f(v)".
3
4  Further, Is there Iteration Algorithm can be use to update
5  policy to gain larger value, which means better policy?
6  Thus, following explore "how to Iterate for better"
7
8  1. Value Iteration
9  2. Policy Iteration
10 3. Truncated Policy Iteration (uniform form of 1 and 2)
11
12 after explore their connections and differences

```

4.0 Summary

▷ Value iteration: it is the iterative algorithm solving the Bellman optimality equation: given an initial value v_0 ,

$$\begin{aligned}
 v_{k+1} &= \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_k) \\
 &\Downarrow \\
 \begin{cases} \text{Policy update: } \pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_k) \\ \text{Value update: } v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k \end{cases}
 \end{aligned}$$

▷ Policy iteration: given an initial policy π_0 ,

$$\begin{cases} \text{Policy evaluation: } v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k} \\ \text{Policy improvement: } \pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_k}) \end{cases}$$

▷ Truncated policy iteration

4.1 Value Iteration

The algorithm (matrix-vector form)

$$v_{k+1} = f(v_k) = \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_k), \quad k = 1, 2, 3 \dots$$

can be decomposed to two steps.

- **Step 1: policy update.** This step is to solve

$$\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_k)$$

where v_k is given.

- **Step 2: value update.**

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$$

Question: is v_k a state value? No, because it is not ensured that v_k satisfies a Bellman equation.

▷ Step 1: Policy update

The elementwise form of

$$\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_k)$$

is

$$\pi_{k+1}(s) = \arg \max_{\pi} \sum_a \pi(a|s) \underbrace{\left(\sum_r p(r|s, a) r + \gamma \sum_{s'} p(s'|s, a) v_k(s') \right)}_{q_k(s, a)}, \quad s \in \mathcal{S}$$

The optimal policy solving the above optimization problem is

$$\pi_{k+1}(a|s) = \begin{cases} 1 & a = a_k^*(s) \\ 0 & a \neq a_k^*(s) \end{cases}$$

where $a_k^*(s) = \arg \max_a q_k(a, s)$. π_{k+1} is called a **greedy policy**, since it simply selects the greatest q-value.

▷ Step 2: Value update

The elementwise form of

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$$

is

$$v_{k+1}(s) = \sum_a \pi_{k+1}(a|s) \underbrace{\left(\sum_r p(r|s, a) r + \gamma \sum_{s'} p(s'|s, a) v_k(s') \right)}_{q_k(s, a)}, \quad s \in \mathcal{S}$$

Since π_{k+1} is greedy, the above equation is simply

$$v_{k+1}(s) = \max_a q_k(a, s)$$

▷ Procedure summary:

$$v_k(s) \rightarrow q_k(s, a) \rightarrow \text{greedy policy } \pi_{k+1}(a|s) \rightarrow \text{new value } v_{k+1} = \max_a q_k(s, a)$$

Pseudocode: Value iteration algorithm

Initialization: The probability model $p(r|s, a)$ and $p(s'|s, a)$ for all (s, a) are known. Initial guess v_0 .

Aim: Search the optimal state value and an optimal policy solving the Bellman optimality equation.

While v_k has not converged in the sense that $\|v_k - v_{k-1}\|$ is greater than a predefined small threshold, for the k th iteration, do

For every state $s \in \mathcal{S}$, do

For every action $a \in \mathcal{A}(s)$, do

$$\text{q-value: } q_k(s, a) = \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_k(s')$$

$$\text{Maximum action value: } a_k^*(s) = \arg \max_a q_k(a, s)$$

$$\text{Policy update: } \pi_{k+1}(a|s) = 1 \text{ if } a = a_k^*, \text{ and } \pi_{k+1}(a|s) = 0 \text{ otherwise}$$

$$\text{Value update: } v_{k+1}(s) = \max_a q_k(a, s)$$

4.2 Policy Iteration

▷ Algorithm description:

Given a random **initial policy** π_0 ,

- **Step 1: policy evaluation (PE)**

This step is to calculate the state value of π_k :

$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$$

Note that v_{π_k} is a state value function.

- **Step 2: policy improvement (PI)**

$$\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_k})$$

The maximization is componentwise!

Remark:

1. in policy iteration step1(PE): v_{π_k} is state value function, while in value iteration is not.

2. Thus, close solution vs iteration solution. In practice, we usually choose an iteration solution
3. So we can find there are two iterations, one is the Algorithm Policy Iteration, another is inner iteration for gain solution of bellman equation
- In policy improvement(PI), π_{k+1} improvement through $q_{\pi_k}(s, a)$, 一般地, state s_i 通过 $a \in \mathcal{A}(s)$ 的Greedy policy 选择 $q(s_i, a)$ 最高的 a , 倘若 $s_i \rightarrow s_j$ 通过 a , 且 s_j 已经在迭代中实现了正确的policy $\pi(a|s_j) = a_{best}$, 那么在本次迭代中, 也会迭代到 s_i 策略正确的policy π 。因而从最靠近目标终点的地方开始实现最优, 再逐步向外迭代变优

Step 1: Policy evaluation

▷ Matrix-vector form: $v_{\pi_k}^{(j+1)} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}^{(j)}$, $j = 0, 1, 2, \dots$

▷ Elementwise form:

$$v_{\pi_k}^{(j+1)}(s) = \sum_a \pi_k(a|s) \left(\sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_{\pi_k}^{(j)}(s') \right), \quad s \in \mathcal{S}$$

Stop when j is sufficiently large or $\|v_{\pi_k}^{(j+1)} - v_{\pi_k}^{(j)}\|$ is sufficiently small.

Step 2: Policy improvement

▷ Matrix-vector form: $\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_k})$

▷ Elementwise form

$$\pi_{k+1}(s) = \arg \max_{\pi} \sum_a \pi(a|s) \underbrace{\left(\sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_{\pi_k}(s') \right)}_{q_{\pi_k}(s, a)}, \quad s \in \mathcal{S}.$$

Here, $q_{\pi_k}(s, a)$ is the action value under policy π_k . Let

$$a_k^*(s) = \arg \max_a q_{\pi_k}(s, a)$$

Then, the greedy policy is

$$\pi_{k+1}(a|s) = \begin{cases} 1 & a = a_k^*(s), \\ 0 & a \neq a_k^*(s). \end{cases}$$

Pseudocode: Policy iteration algorithm

Initialization: The probability model $p(r|s, a)$ and $p(s'|s, a)$ for all (s, a) are known. Initial guess π_0 .

Aim: Search for the optimal state value and an optimal policy.

While v_{π_k} has not converged, for the k th iteration, do

Policy evaluation:

Initialization: an arbitrary initial guess $v_{\pi_k}^{(0)}$

While $v_{\pi_k}^{(j)}$ has not converged, for the j th iteration, do

For every state $s \in \mathcal{S}$, do

$$v_{\pi_k}^{(j+1)}(s) = \sum_a \pi_k(a|s) \left[\sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_{\pi_k}^{(j)}(s') \right]$$

Policy improvement:

For every state $s \in \mathcal{S}$, do

For every action $a \in \mathcal{A}$, do

$$q_{\pi_k}(s, a) = \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_{\pi_k}(s')$$

$$a_k^*(s) = \arg \max_a q_{\pi_k}(s, a)$$

$$\pi_{k+1}(a|s) = 1 \text{ if } a = a_k^*, \text{ and } \pi_{k+1}(a|s) = 0 \text{ otherwise}$$

4.3 Value vs Policy Iteration and Truncated Policy Iteration

Policy iteration: start from π_0

- Policy evaluation (PE):

$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$$

- Policy improvement (PI):

$$\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_k})$$

Value iteration: start from v_0

- Policy update (PU):

$$\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_k)$$

- Value update (VU):

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$$

▷ The two algorithms are very similar:

$$\text{Policy iteration: } \pi_0 \xrightarrow{PE} v_{\pi_0} \xrightarrow{PI} \pi_1 \xrightarrow{PE} v_{\pi_1} \xrightarrow{PI} \pi_2 \xrightarrow{PE} v_{\pi_2} \xrightarrow{PI} \dots$$

$$\text{Value iteration: } u_0 \xrightarrow{PU} \pi'_1 \xrightarrow{VU} u_1 \xrightarrow{PU} \pi'_2 \xrightarrow{VU} u_2 \xrightarrow{PU} \dots$$

PE=policy evaluation. PI=policy improvement.

PU=policy update. VU=value update.

Remark:

1. We can find that: **in Policy Iteration, PE through iteration to solve bellman equation for v_{π_k} , which can be considered as a large number of VU in Value Iteration**
2. Thus, in Policy Iteration, v_{π_k} is more precise than value iteration. Through experiments, we can find Policy Iteration iterate less time for coverage, but in each Iteration, PE has low-level iteration ;)
3. So Truncated Policy Iteration is one compromise, which choose finite numbers of low-level iteration VU to solve bellman equation for v_{π_k}

Consider the step of solving $v_{\pi_1} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}$:

$$v_{\pi_1}^{(0)} = v_0$$

$$\text{value iteration} \leftarrow v_1 \leftarrow v_{\pi_1}^{(1)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(0)}$$

$$v_{\pi_1}^{(2)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(1)}$$

⋮

$$\text{truncated policy iteration} \leftarrow \bar{v}_1 \leftarrow v_{\pi_1}^{(j)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(j-1)}$$

⋮

$$\text{policy iteration} \leftarrow v_{\pi_1} \leftarrow v_{\pi_1}^{(\infty)} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(\infty)}$$

- The value iteration algorithm computes *once*.
- The policy iteration algorithm computes *an infinite number of iterations*.
- The **truncated policy iteration algorithm** computes *a finite number of iterations* (say j). The rest iterations from j to ∞ are truncated.

Pseudocode: Truncated policy iteration algorithm

Initialization: The probability model $p(r|s, a)$ and $p(s'|s, a)$ for all (s, a) are known. Initial guess π_0 .

Aim: Search for the optimal state value and an optimal policy.

While v_k has not converged, for the k th iteration, do

Policy evaluation:

 Initialization: select the initial guess as $v_k^{(0)} = v_{k-1}$. The maximum iteration is set to be j_{truncate} .

 While $j < j_{\text{truncate}}$, do

 For every state $s \in \mathcal{S}$, do

$$v_k^{(j+1)}(s) = \sum_a \pi_k(a|s) \left[\sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_k^{(j)}(s') \right]$$

 Set $v_k = v_k^{(j_{\text{truncate}})}$

Policy improvement:

 For every state $s \in \mathcal{S}$, do

 For every action $a \in \mathcal{A}(s)$, do

$$q_k(s, a) = \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_k(s')$$

$$a_k^*(s) = \arg \max_a q_k(s, a)$$

$$\pi_{k+1}(a|s) = 1 \text{ if } a = a_k^*, \text{ and } \pi_{k+1}(a|s) = 0 \text{ otherwise}$$

5、Monte Carlo Methods 【from model-calculation to data-estimation】

Code block

```
1  In above 4、Iteration-Algorithm, we have supposed have model(environment)
   distribution
2  that  $p(r|s,a)$  and  $p(s|s,a)$ 
3
4  However, in real-sense situations, environment is often difficult to model.
5
6  Thus, we want to explore the possibility of RL without environment-model based
   on above
7  policy iteration algorithm.
8  Though we don't have environment, we must have datas, which should be large
   enough
9  that can convert environment information.
10
11  So, Let's say Model or Data!
```

5.0 Summary

▷ Generalized policy iteration:

- Not a specific algorithm.
- It refers to the general idea or framework of switching between [policy-evaluation](#) and [policy-improvement](#) processes.
- Many RL algorithms fall into this framework.

▷ Summary:

- Monte Carlo estimation refers to a broad class of techniques that rely on repeated random sampling to solve approximation problems.
- Why we care about Monte Carlo estimation? Because it does not require the model!
- Why we care about mean estimation? Because state value and action value are defined as expectations of random variables!

Key points:

- Mean estimation by the Monte Carlo methods
- Three algorithms:
 - MC Basic
 - MC Exploring Starts
 - MC ϵ -Greedy
- Relationship among the three algorithms
- Optimality vs exploration of ϵ -greedy policies

5.1 Math Theory

Law of Large Numbers

For a random variable X , suppose $\{x_j\}_{j=1}^N$ are some iid samples. Let $\bar{x} = \frac{1}{N} \sum_{j=1}^N x_j$ be the average of the samples. Then,

$$\mathbb{E}[\bar{x}] = \mathbb{E}[X],$$

$$\text{Var}[\bar{x}] = \frac{1}{N} \text{Var}[X].$$

As a result, $\bar{x}$ is an unbiased estimate of $\mathbb{E}[X]$ and its variance decreases to zero as N increases to infinity.

▷ The samples must be iid (independent and identically distributed)

Remark:

- It indicates the transition from probability to statistics
- Thus, we can convert from calculating model expectation to statistical data mean necessary about environment (here is action value, which acan be used to improve policies directly, yet state value cannot)

Policy iteration has two steps in each iteration:

$$\begin{cases} \text{Policy evaluation: } v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k} \\ \text{Policy improvement: } \pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_k}) \end{cases}$$

The elementwise form of the **policy improvement step** is:

$$\begin{aligned} \pi_{k+1}(s) &= \arg \max_{\pi} \sum_a \pi(a|s) \left[\sum_r p(r|s, a) r + \gamma \sum_{s'} p(s'|s, a) v_{\pi_k}(s') \right] \\ &= \arg \max_{\pi} \sum_a \pi(a|s) q_{\pi_k}(s, a), \quad s \in \mathcal{S} \end{aligned}$$

The key is to calculate $q_{\pi_k}(s, a)$!

Two expressions of action value:

- Expression 1 requires the model:

$$q_{\pi_k}(s, a) = \sum_r p(r|s, a) r + \gamma \sum_{s'} p(s'|s, a) v_{\pi_k}(s')$$

- Expression 2 does not require the model:

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$$

Idea to achieve model-free RL: We can use expression 2 to obtain $q_{\pi_k}(s, a)$ based on *data (samples or experiences)*!

5.2 model-free policy iteration (MC Basic)

How to obtain $q_{\pi_k}(s, a)$ based on data?

- Starting from (s, a) , following policy π_k , generate an episode.
- The return of this episode is $g(s, a)$, which is a sample of G_t in

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$$

- Suppose we have a set of episodes and hence $\{g^{(j)}(s, a)\}$. Then,

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a] \approx \frac{1}{N} \sum_{i=1}^N g^{(i)}(s, a).$$

Fundamental idea: When model is unavailable, we can use data.

Given an initial policy π_0 , there are two steps in the k th iteration.

- **Step 1: policy evaluation.** This step aims to estimate $q_{\pi_k}(s, a)$ for all (s, a) . Specifically, for each (s, a) , run sufficiently many episodes. The average of their returns, denoted as $q_k(s, a)$, is used to approximate $q_{\pi_k}(s, a)$.
 - The first step of the *policy iteration algorithm* calculates $q_{\pi_k}(s, a)$ by firstly solving the state values v_{π_k} from the Bellman equation. This requires the model.
 - The first step of the *MC Basic algorithm* is to directly estimate $q_k(s, a)$ from experiences samples. This does not require the model.
- **Step 2: policy improvement.** This step aims to solve $\pi_{k+1}(s) = \arg \max_{\pi} \sum_a \pi(a|s) q_k(s, a)$ for all $s \in \mathcal{S}$. The greedy optimal policy is $\pi_{k+1}(a_k^*|s) = 1$ where $a_k^* = \arg \max_a q_k(s, a)$.
 - This step is exactly the same as the second step of the policy iteration algorithm.

Pseudocode: MC Basic algorithm (a model-free variant of policy iteration)

Initialization: Initial guess π_0 .

Aim: Search for an optimal policy.

For the k th iteration ($k = 0, 1, 2, \dots$), do

For every state $s \in \mathcal{S}$, do

For every action $a \in \mathcal{A}(s)$, do

Collect sufficiently many episodes starting from (s, a) following π_k

Policy evaluation:

$q_{\pi_k}(s, a) \approx q_k(s, a) =$ average return of all the episodes starting from (s, a)

Policy improvement:

$a_k^*(s) = \arg \max_a q_k(s, a)$

$\pi_{k+1}(a|s) = 1$ if $a = a_k^*$, and $\pi_{k+1}(a|s) = 0$ otherwise

Remark:

- The model-free algorithm is built up based on model-based ones, just from probability to statistics
- Yet the MC basic method is inefficient, so it cannot be used in practices

5.3 MC Exploring Starts

Code block

1 To use data more efficiently, evolved from basic MC

5.3.1 About "visit" for data efficiency

▷ Consider a grid-world example, following a policy π , we can get an episode such as

$s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots$

▷ **Visit:** every time a state-action pair appears in the episode, it is called a *visit* of that state-action pair.

▷ Methods to use the data: **Initial-visit method**

- Just calculate the return and approximate $q_\pi(s_1, a_2)$.
- This is what the MC Basic algorithm does.
- Disadvantage: Not fully utilize the data.

▷ The episode also visits other state-action pairs.

$s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots$ [original episode]
 $s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots$ [episode starting from (s_2, a_4)]
 $s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots$ [episode starting from (s_1, a_2)]
 $s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots$ [episode starting from (s_2, a_3)]
 $s_5 \xrightarrow{a_1} \dots$ [episode starting from (s_5, a_1)]

Can estimate $q_\pi(s_1, a_2)$, $q_\pi(s_2, a_4)$, $q_\pi(s_2, a_3)$, $q_\pi(s_5, a_1), \dots$

Data-efficient methods:

- first-visit method
- every-visit method

Remark:

- First visit: the method only uses first appear visit in one episode (eg. $q(s_1, a_2)$ above appear twice, but only use first visit)
- Every visit: use every visit in one episode (how to assemble them can be communicated? Mean directly/ discounted factor / eva update real-time (the eva also can change by time)?)

5.3.2 About policy improvement frequency

- ▷ Another aspect in MC-based RL is **when to update the policy**. There are two methods.
- **The first method** is, in the policy evaluation step, to collect **all** the episodes starting from a state-action pair and then use the average return to approximate the action value.
 - This is the one adopted by the MC Basic algorithm.
 - The problem of this method is that the agent has to **wait** until all episodes have been collected.
 - **The second method** uses the return of a **single** episode to approximate the action value.
 - In this way, we can improve the policy **episode-by-episode**.

4.3.3 MC exploring Starts method

Algorithm: MC Exploring Starts (an efficient variant of MC Basic)

Initialization: Initial policy $\pi_0(a|s)$ and initial value $q(s, a)$ for all (s, a) . $\text{Returns}(s, a) = 0$ and $\text{Num}(s, a) = 0$ for all (s, a) .

Goal: Search for an optimal policy.

For each episode, do

Episode generation: Select a starting state-action pair (s_0, a_0) and ensure that all pairs can be possibly selected (this is the exploring-starts condition). Following the current policy, generate an episode of length T : $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$.

Initialization for each episode: $g \leftarrow 0$

For each step of the episode, $t = T - 1, T - 2, \dots, 0$, do

$g \leftarrow \gamma g + r_{t+1}$

$\text{Returns}(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) + g$

$\text{Num}(s_t, a_t) \leftarrow \text{Num}(s_t, a_t) + 1$

Policy evaluation:

$q(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) / \text{Num}(s_t, a_t)$

Policy improvement:

$\pi(a|s_t) = 1$ if $a = \arg \max_a q(s_t, a)$ and $\pi(a|s_t) = 0$ otherwise

Remark:

1. The above algorithm computes Return g from tail to head is one trick for compute efficiency
2. For data efficiency, it is every-visit method, and deal all same visit by mean directly
3. For when to update policy, it updates policy each step, not each episode like above (if needed, we can set only $T=0$ update)

Pseudocode: MC Exploring Starts (a sample-efficient variant of MC Basic)

Initialization: Initial guess π_0 .

Aim: Search for an optimal policy.

For each episode, do

Episode generation: Randomly select a starting state-action pair (s_0, a_0) and ensure that all pairs can be possibly selected. Following the current policy, generate an episode of length T : $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$.

Policy evaluation and policy improvement:

Initialization: $g \leftarrow 0$

For each step of the episode, $t = T - 1, T - 2, \dots, 0$, do

$g \leftarrow \gamma g + r_{t+1}$

Use the first-visit method:

If (s_t, a_t) does not appear in $(s_0, a_0, s_1, a_1, \dots, s_{t-1}, a_{t-1})$, then

$Returns(s_t, a_t) \leftarrow Returns(s_t, a_t) + g$

$q(s_t, a_t) = \text{average}(Returns(s_t, a_t))$

$\pi(a|s_t) = 1$ if $a = \arg \max_a q(s_t, a)$

first-visit version

5.4 MC ϵ - Greedy

Code block

- 1 when we improve policy above, always choose the action which map max action value q .
- 2
- 3 In theory, iteration Algorithm needs it to explore all state-action visit pair,
- 4 Yet in practice, deterministics update may limit exploration, which maybe miss some pair
- 5
- 6 Thus, we choose soft policy (each action possibility is positive) to ensure exploration
- 7
- 8 Here, is ϵ - Greedy. There may be more tricks in soft policies

▷ What is the ε -greedy policy?

$$\pi(a|s) = \begin{cases} 1 - \frac{\varepsilon}{|\mathcal{A}(s)|}(|\mathcal{A}(s)| - 1), & \text{for the greedy action,} \\ \frac{\varepsilon}{|\mathcal{A}(s)|}, & \text{for the other } |\mathcal{A}(s)| - 1 \text{ actions.} \end{cases}$$

where $\varepsilon \in [0, 1]$ and $|\mathcal{A}(s)|$ is the number of actions for s .

- Example: if $\varepsilon = 0.2$, then

$$\frac{\varepsilon}{|\mathcal{A}(s)|} = \frac{0.2}{5} = 0.04, \quad 1 - \frac{\varepsilon}{|\mathcal{A}(s)|}(|\mathcal{A}(s)| - 1) = 1 - 0.04 \times 4 = 0.84$$

- The chance to choose the greedy action is always greater than other actions, because

$$1 - \frac{\varepsilon}{|\mathcal{A}(s)|}(|\mathcal{A}(s)| - 1) = 1 - \varepsilon + \frac{\varepsilon}{|\mathcal{A}(s)|} \geq \frac{\varepsilon}{|\mathcal{A}(s)|}$$

ε -greedy policies can balance **exploitation and exploration**.

- When $\varepsilon \rightarrow 0$, it becomes greedy!

$$\pi(a|s) = \begin{cases} 1 - \frac{\varepsilon}{|\mathcal{A}(s)|}(|\mathcal{A}(s)| - 1) = 1, & \text{for the greedy action,} \\ \frac{\varepsilon}{|\mathcal{A}(s)|} = 0, & \text{for the other } |\mathcal{A}(s)| - 1 \text{ actions.} \end{cases}$$

More exploitation but less exploration.

- When $\varepsilon \rightarrow 1$, it becomes a uniform distribution.

$$\pi(a|s) = \begin{cases} 1 - \frac{\varepsilon}{|\mathcal{A}(s)|}(|\mathcal{A}(s)| - 1) = \frac{1}{|\mathcal{A}(s)|}, & \text{for the greedy action,} \\ \frac{\varepsilon}{|\mathcal{A}(s)|} = \frac{1}{|\mathcal{A}(s)|}, & \text{for the other } |\mathcal{A}(s)| - 1 \text{ actions.} \end{cases}$$

More exploration but less exploitation.

Now, the policy improvement step is changed to solve

$$\pi_{k+1}(s) = \arg \max_{\pi \in \Pi_\varepsilon} \sum_a \pi(a|s) q_{\pi_k}(s, a),$$

where Π_ε denotes the set of all ε -greedy policies with a fixed value of ε . The optimal policy here is

$$\pi_{k+1}(a|s) = \begin{cases} 1 - \frac{|\mathcal{A}(s)| - 1}{|\mathcal{A}(s)|} \varepsilon, & a = a_k^*, \\ \frac{1}{|\mathcal{A}(s)|} \varepsilon, & a \neq a_k^*. \end{cases}$$

Summary:

- MC ε -Greedy is the *same* as that of MC Exploring Starts *except* that the former uses ε -greedy policies.
- It does not require exploring starts, but still requires to visit all state-action pairs in a different form.

Remark:

- Without require exploring starts 's reason is that there is more exploration, it can be another start to "more start"

Algorithm: MC ϵ -Greedy (a variant of MC Exploring Starts)

Initialization: Initial policy $\pi_0(a|s)$ and initial value $q(s, a)$ for all (s, a) . $\text{Returns}(s, a) = 0$ and $\text{Num}(s, a) = 0$ for all (s, a) . $\epsilon \in (0, 1]$

Goal: Search for an optimal policy.

For each episode, do

Episode generation: Select a starting state-action pair (s_0, a_0) (the exploring starts condition is not required). Following the current policy, generate an episode of length T : $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$.

Initialization for each episode: $g \leftarrow 0$

For each step of the episode, $t = T - 1, T - 2, \dots, 0$, do

$g \leftarrow \gamma g + r_{t+1}$

$\text{Returns}(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) + g$

$\text{Num}(s_t, a_t) \leftarrow \text{Num}(s_t, a_t) + 1$

Policy evaluation:

$q(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) / \text{Num}(s_t, a_t)$

Policy improvement:

Let $a^* = \arg \max_a q(s_t, a)$ and

$$\pi(a|s_t) = \begin{cases} 1 - \frac{|\mathcal{A}(s_t)| - 1}{|\mathcal{A}(s_t)|} \epsilon, & a = a^* \\ \frac{1}{|\mathcal{A}(s_t)|} \epsilon, & a \neq a^* \end{cases}$$

5.5 Comparison among later-two algorithms

- The **advantage** of ϵ -greedy policies is that they have strong exploration ability when ϵ is large.
 - Then, the exploring starts condition is not required.
- The **disadvantage** is that ϵ -greedy policies are **not optimal** in general.
 - It is only optimal in the set Π_ϵ of all ϵ -greedy policies.
 - ϵ cannot be too large. We can also use a decaying ϵ .

Remark:

- usually, when ϵ larger, the final gained π_ϵ would worse
- Decaying trick

6、Stochastic Approximation

Code block

- 1 In Monte Carlo methods, we use mean estimation from sample datas
- 2 instead of expectation calculation from environment model.

- 3
- 4 To estimate mean, we need sample N samples, and then N divides sum of all samples
- 5 Yet, this method is inefficient, we have to wait all samples finished to get estimation
- 6
- 7 Here we introduce iteration algorithm for efficient mean estimation from samples

6.0 Summary

- Mean estimation: compute $\mathbb{E}[X]$ using $\{x_k\}$

$$w_{k+1} = w_k - \frac{1}{k}(w_k - x_k).$$

- RM algorithm: solve $g(w) = 0$ using $\{\tilde{g}(w_k, \eta_k)\}$

$$w_{k+1} = w_k - a_k \tilde{g}(w_k, \eta_k)$$

- SGD algorithm: minimize $J(w) = \mathbb{E}[f(w, X)]$ using $\{\nabla_w f(w_k, x_k)\}$

$$w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k),$$

These results are useful:

- We will see in the next chapter that the temporal-difference learning algorithms can be viewed as stochastic approximation algorithms and hence have similar expressions.
- They are important optimization techniques that can be applied to many other fields.

6.1 Motivation from mean estimation

In particular, suppose

$$w_{k+1} = \frac{1}{k} \sum_{i=1}^k x_i \quad k = 1, 2, \dots$$

and hence

$$w_k = \frac{1}{k-1} \sum_{i=1}^{k-1} x_i, \quad k = 2, 3, \dots$$

Then, w_{k+1} can be expressed in terms of w_k as

$$\begin{aligned} w_{k+1} &= \frac{1}{k} \sum_{i=1}^k x_i = \frac{1}{k} \left(\sum_{i=1}^{k-1} x_i + x_k \right) \\ &= \frac{1}{k} ((k-1)w_k + x_k) = w_k - \frac{1}{k}(w_k - x_k). \end{aligned}$$

Therefore, we obtain the following iterative algorithm:

$$w_{k+1} = w_k - \frac{1}{k}(w_k - x_k).$$

Remarks about this algorithm:

$$w_{k+1} = w_k - \frac{1}{k}(w_k - x_k).$$

- An **advantage** of this algorithm is that it is **incremental**. A mean estimate can be obtained immediately once a sample is received. Then, the mean estimate can be used for other purposes immediately.
- The mean estimate is not accurate in the beginning due to insufficient samples (that is $w_k \neq \mathbb{E}[X]$). However, **it is better than nothing**. As more samples are obtained, the estimate can be improved gradually (that is $w_k \rightarrow \mathbb{E}[X]$ as $k \rightarrow \infty$).

6.2 Robbins-Monro Algorithm

6.2.1 What is Stochastic approximation (SA)

Stochastic approximation (SA):

- SA refers to a broad class of stochastic iterative algorithms solving root finding or optimization problems.
- Compared to many other root-finding algorithms such as gradient-based methods, SA is powerful in the sense that **it does not require to know the expression of the objective function nor its derivative**.

Remark: **Not required to know the expression means model-free**

6.2.2 Robbins-Monro Algorithm

6.2.2.1 Connections among algorithms

- RM algorithm is a pioneering work in the field of SA.
- The stochastic gradient descent algorithm is a special form of RM

6.2.2.2 RM application set up

Problem statement: Suppose we would like to find the root of the equation

$$g(w) = 0,$$

where $w \in \mathbb{R}$ is the variable to be solved and $g : \mathbb{R} \rightarrow \mathbb{R}$ is a function.

- Many problems can be eventually converted to this root finding problem. For example, suppose $J(w)$ is an objective function to be minimized. Then, the optimization problem can be converted to

$$g(w) = \nabla_w J(w) = 0$$

- Note that an equation like $g(w) = c$ with c as a constant can also be converted to the above equation by rewriting $g(w) - c$ as a new function.

Remark: RM is designed for model-free calculation of the root of the expression-unknown equation $g(w) = 0$

The Robbins-Monro (RM) algorithm can solve this problem:

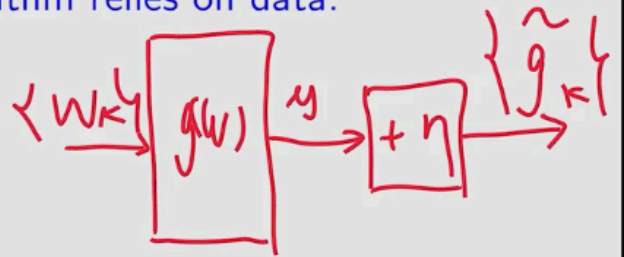
$$w_{k+1} = w_k - a_k \tilde{g}(w_k, \eta_k), \quad k = 1, 2, 3, \dots$$

where

- w_k is the k th estimate of the root
- $\tilde{g}(w_k, \eta_k) = g(w_k) + \eta_k$ is the k th noisy observation
- a_k is a positive coefficient.

The function $g(w)$ is a **black box**! This algorithm relies on data:

- Input sequence: $\{w_k\}$
- Noisy output sequence: $\{\tilde{g}(w_k, \eta_k)\}$



Philosophy: without model, we need data!

Theorem (Robbins-Monro Theorem)

In the Robbins-Monro algorithm, if

- 1) $0 < c_1 \leq \nabla_w g(w) \leq c_2$ for all w ;
- 2) $\sum_{k=1}^{\infty} a_k = \infty$ and $\sum_{k=1}^{\infty} a_k^2 < \infty$;
- 3) $\mathbb{E}[\eta_k | \mathcal{H}_k] = 0$ and $\mathbb{E}[\eta_k^2 | \mathcal{H}_k] < \infty$;

where $\mathcal{H}_k = \{w_k, w_{k-1}, \dots\}$, then w_k converges with probability 1 (w.p.1) to the root w^* satisfying $g(w^*) = 0$.

Explanation of the three conditions:

- **Condition 1:** $0 < c_1 \leq \nabla_w g(w) \leq c_2$ for all w
 - g should be **monotonically increasing**, which ensures that the root of $g(w) = 0$ exists and is unique
 - The gradient is bounded from the above.
 - This condition is not strict. Consider the example $g(w) = \nabla_w J(w) = 0$. This condition requires that $g(w)$ is convex.
- **Condition 2:** $\sum_{k=1}^{\infty} a_k = \infty$ and $\sum_{k=1}^{\infty} a_k^2 < \infty$
 - $\sum_{k=1}^{\infty} a_k^2 < \infty$ ensures that a_k converges to zero as $k \rightarrow \infty$.
 - $\sum_{k=1}^{\infty} a_k = \infty$ ensures that a_k do not converge to zero too fast.
- **Condition 3:** $\mathbb{E}[\eta_k | \mathcal{H}_k] = 0$ and $\mathbb{E}[\eta_k^2 | \mathcal{H}_k] < \infty$
 - A special yet common case is that $\{\eta_k\}$ is an iid stochastic sequence satisfying $\mathbb{E}[\eta_k] = 0$ and $\mathbb{E}[\eta_k^2] < \infty$. The observation error η_k is not required to be Gaussian.

We will see that a_k is often selected as a **sufficiently small constant** in many RL algorithms. Although the second condition is not satisfied in this case, the algorithm can still work effectively.

Remark: when $\{a_k\} \rightarrow 0$ as $k \rightarrow \infty$, the effect of later data decreases, which means the later data works less importantly. Yet in practice, we may still have a belief in later data.

6.2.2.3 Optional Theory

Theorem (Dvoretzky's Theorem)

Consider a stochastic process

$$w_{k+1} = (1 - \alpha_k)w_k + \beta_k \eta_k,$$

where $\{\alpha_k\}_{k=1}^{\infty}, \{\beta_k\}_{k=1}^{\infty}, \{\eta_k\}_{k=1}^{\infty}$ are stochastic sequences. Here $\alpha_k \geq 0, \beta_k \geq 0$ for all k . Then, w_k would converge to zero with probability 1 if the following conditions are satisfied:

- 1) $\sum_{k=1}^{\infty} \alpha_k = \infty, \sum_{k=1}^{\infty} \alpha_k^2 < \infty; \sum_{k=1}^{\infty} \beta_k^2 < \infty$ uniformly w.p.1;
- 2) $\mathbb{E}[\eta_k | \mathcal{H}_k] = 0$ and $\mathbb{E}[\eta_k^2 | \mathcal{H}_k] \leq C$ w.p.1;

where $\mathcal{H}_k = \{w_k, w_{k-1}, \dots, \eta_{k-1}, \dots, \alpha_{k-1}, \dots, \beta_{k-1}, \dots\}$.

- A more general result than the RM theorem.
 - It can be used to prove the RM theorem
 - It can be used to analyze the mean estimation problem.
 - An extension of it can be used to analyze Q-learning and TD learning algorithms.

6.2.3 Stochastic gradient descent

6.2.3.1 Application Problem set up

Problem setup: Suppose we aim to solve the following **optimization problem**:

$$\min_w J(w) = \mathbb{E}[f(w, X)]$$

- w is the parameter to be optimized.
- X is a random variable. The expectation is with respect to X .
- w and X can be either scalars or vectors. The function $f(\cdot)$ is a scalar.

Remark:

- X is a random variable, whose distribution exists, yet it is difficult (maybe unable) to express in practice
- Optimization Problem compares with Deeping Learning
 - The above optimization problem can be described as: we have a function $f(x; w)$, where x is input, w is function's weight. X is our training dataset. The above optimization problem is finding the best weight w for the dataset X , which can get minimum from f

- To align with the above description, Deep Learning still uses similar parameters. We have a neural network $\hat{y} = f(x; w)$ (where w is the network's weight), which can also be considered as one complex function. To fulfill one purpose that we want f has some abilities, we will design one loss function $L(y, \hat{y}) = L(y, f(x; w))$. Then we'd like to minimize the loss $w = \operatorname{argmin}_w L(y, \hat{y})$. Our training dataset is (X, Y) . Compared with optimization problem, optimization problem's scalar is f itself, while deep learning is L and indirectly affects f (improves w)

6.2.3.2 Minimize $J(w) = \mathbb{E}(f(X; w))$ Gradient Methods

Code block

- 1 Here gradient descent idea is directly from gradient math definition
- 2 $J(w)$ will decrease if w goes along the gradient direction

Method 1: gradient descent (GD)

$$w_{k+1} = w_k - \alpha_k \nabla_w \mathbb{E}[f(w_k, X)] = w_k - \alpha_k \mathbb{E}[\nabla_w f(w_k, X)]$$

Drawback: the expected value is difficult to obtain.

Method 2: batch gradient descent (BGD)

$$\mathbb{E}[\nabla_w f(w_k, X)] \approx \frac{1}{n} \sum_{i=1}^n \nabla_w f(w_k, x_i).$$

$$w_{k+1} = w_k - \alpha_k \frac{1}{n} \sum_{i=1}^n \nabla_w f(w_k, x_i).$$

Drawback: it requires many samples in each iteration for each w_k .

Method 3: stochastic gradient descent (SGD)

$$\underline{w_{k+1}} = \underline{w_k} - \underline{\alpha_k} \underline{\nabla_w f(w_k, x_k)},$$

- Compared to the gradient descent method: Replace the true gradient $\mathbb{E}[\nabla_w f(w_k, X)]$ by the stochastic gradient $\nabla_w f(w_k, x_k)$.
- Compared to the batch gradient descent method: let $n = 1$.

Remark:

- Here GD means calculate $\mathbb{E}$ through model (X distribution)
- Here BGD inavailable means sample all data ? (maybe OK for batch not for whole)

6.2.3.3 SGD back to RM

Code block

- As we know, find one function f 's excited minimum value, necessary to find f differential
- f' zero point
-
- Thus, minimize $J(w) \rightarrow$ find $\nabla J(w) = 0$

Remark:

- find w to minimize $J(w) \Rightarrow$ find w to fulfill $\nabla J(w) = 0$
- Thus, we come back to RM Algorithm !!

What we can measure is

$$\begin{aligned} \tilde{g}(w, \eta) &= \nabla_w f(w, x) \\ &= \underbrace{\mathbb{E}[\nabla_w f(w, X)]}_{g(w)} + \underbrace{\nabla_w f(w, x) - \mathbb{E}[\nabla_w f(w, X)]}_{\eta} \end{aligned}$$

Then, the RM algorithm for solving $g(w) = 0$ is

$$\underline{w_{k+1}} = \underline{w_k} - \underline{a_k} \underline{\tilde{g}(w_k, \eta_k)} = w_k - a_k \nabla_w f(w_k, x_k).$$

- It is exactly the SGD algorithm.
- Therefore, SGD is a special RM algorithm.

Since SGD is a special RM algorithm, its convergence naturally follows.

Theorem (Convergence of SGD)

In the SGD algorithm, if

- $0 < c_1 \leq \nabla_w^2 f(w, X) \leq c_2$;
- $\sum_{k=1}^{\infty} a_k = \infty$ and $\sum_{k=1}^{\infty} a_k^2 < \infty$;
- $\{x_k\}_{k=1}^{\infty}$ is iid;

then $\underline{w_k}$ converges to the root of $\underline{\nabla_w \mathbb{E}[f(w, X)] = 0}$ with probability 1.

Remrak: 【缺乏多维矩阵的知识】

- c_1 c_2 matrixes too as w is one matrix, which indicates f ?
- $\nabla_w^2 f(X; w)$ indicates f is convex function, convexity matrix ? Convex optimization?

6.2.3.4 SGD convergence pattern: stochastic gradient's error

Note that

$$\delta_k \leq \frac{\underbrace{|\nabla_w f(w_k, x_k) - \mathbb{E}[\nabla_w f(w_k, X)]|}_{\substack{\text{stochastic gradient} \quad \text{true gradient}}} }{\underbrace{c|w_k - w^*|}_{\text{distance to the optimal solution}}}.$$

Handwritten notes: w_k^ (crossed out), w_{1c} (crossed out), and arrows pointing to the stochastic and true gradient terms.*

The above equation suggests an interesting convergence pattern of SGD.

- The relative error δ_k is inversely proportional to $|w_k - w^*|$.
- When $|w_k - w^*|$ is large, δ_k is small and SGD behaves like GD.
- When w_k is close to w^* , the relative error may be large and the convergence exhibits more randomness in the neighborhood of w^* .

6.2.3.5 About "Not variant" determinisitc situation's SGD

- The formulation of SGD we introduced above involves random variables and expectation.
- One may often encounter a **deterministic** formulation of SGD without involving any random variables.

Consider the optimization problem:

$$\min_w \quad \underline{J(w)} = \frac{1}{n} \sum_{i=1}^n f(w, x_i), \quad \leftarrow$$

- $f(w, x_i)$ is a parameterized function.
- w is the parameter to be optimized.
- a set of real numbers $\{x_i\}_{i=1}^n$, where x_i does not have to be a sample of any random variable.

The gradient descent algorithm for solving this problem is

$$w_{k+1} = w_k - \alpha_k \nabla_w J(w_k) = w_k - \alpha_k \frac{1}{n} \sum_{i=1}^n \nabla_w f(w_k, x_i).$$

Suppose the set is large and we can only fetch a single number every time. In this case, we can use the following iterative algorithm:

$$w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k).$$

Questions:

- Is this algorithm SGD? It does not involve any random variables or expected values.
 - How should we use the finite set of numbers $\{x_i\}_{i=1}^n$? Should we sort these numbers in a certain order and then use them one by one? Or should we randomly sample a number from the set?
- Handwritten note: "xix" with an arrow pointing to the set of numbers.*

A quick answer to the above questions is that we can introduce a random variable manually and convert the *deterministic formulation* to the *stochastic formulation* of SGD.

In particular, suppose X is a random variable defined on the set $\{x_i\}_{i=1}^n$. Suppose its probability distribution is uniform such that

$$p(X = x_i) = 1/n$$

Then, the deterministic optimization problem becomes a stochastic one:

$$\min_w J(w) = \frac{1}{n} \sum_{i=1}^n f(w, x_i) = \mathbb{E}[f(w, X)].$$

- The last equality in the above equation is strict instead of approximate. Therefore, the algorithm is SGD.
- The estimate converges if x_k is *uniformly* and independently sampled from $\{x_i\}_{i=1}^n$. x_k may repeatedly take the same number in $\{x_i\}_{i=1}^n$ since it is sampled randomly.

Remark:

- In practice, here $\{x_i\}_{i=1}^n$ can be considered as the dataset X
- We manually introduce this uniform random variable, so $\frac{1}{n} \rightarrow \mathbb{E}$.
- Thus, sample operation should follow the uniform distribution

6.2.4 BGD vs MBGD vs SGD

Suppose we would like to minimize $J(w) = \mathbb{E}[f(w, X)]$ given a set of random samples $\{x_i\}_{i=1}^n$ of X . The BGD, SGD, MBGD algorithms solving this problem are, respectively,

$$w_{k+1} = w_k - \alpha_k \frac{1}{n} \sum_{i=1}^n \nabla_w f(w_k, x_i), \quad (\text{BGD})$$

$$w_{k+1} = w_k - \alpha_k \frac{1}{m} \sum_{j \in \mathcal{I}_k} \nabla_w f(w_k, x_j), \quad (\text{MBGD})$$

$$w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k). \quad (\text{SGD})$$

- **In the BGD algorithm**, all the samples are used in every iteration. When n is large, $(1/n) \sum_{i=1}^n \nabla_w f(w_k, x_i)$ is close to the true gradient $\mathbb{E}[\nabla_w f(w_k, X)]$.
- **In the MBGD algorithm**, $\mathcal{I}_k$ is a subset of $\{1, \dots, n\}$ with the size as $|\mathcal{I}_k| = m$. The set $\mathcal{I}_k$ is obtained by m times iid samplings.
- **In the SGD algorithm**, x_k is randomly sampled from $\{x_i\}_{i=1}^n$ at time k .

Remark:

- Here MBGD is 随机批次梯度下降, which is used widely in DL.

7、Temporal-Difference Learning

Code block

```
1  RM algorithm can be considered as one method for solution of equation
2
3  In MC, policy evaluation need calculate Return G of one whole episode.
4  However, TD through RM algorithm evaluate policy (solve Bellman equation)
   online
```

7.0 Summary

- Value v 、action-value q 的TD estimation估计方法: 即利用RM的变形解BE方程:
- $q_{t+1} = q_t + \alpha_t * (TD_error)$
Q-learning对应解BOE, 在max中不依赖target policy π , 所以可以是off-policy的

7.1 State Value estimation

7.1.1 Problem Setting

- Given policy π , the aim is to estimate the state value $\{v_\pi(s)\}_{s \in \mathcal{S}}$ under π
- Experience samples: $\{(s_t, r_{t+1}, s_{t+1})\}_t$ generated by π
- Value estimation: $v_t(s_t) \rightarrow v_\pi(s_t)$ 表示 t 时刻对current state s_t 的current value estimation v_t

7.1.2 Value Estimation TD Algorithm

The TD learning algorithm is

$$v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t) [v_t(s_t) - [r_{t+1} + \gamma v_t(s_{t+1})]], \quad (1)$$

$$v_{t+1}(s) = v_t(s), \quad \forall s \neq s_t, \quad (2)$$

where $t = 0, 1, 2, \dots$

The TD algorithm can be annotated as

$$\underbrace{v_{t+1}(s_t)}_{\text{new estimate}} = \underbrace{v_t(s_t)}_{\text{current estimate}} - \alpha_t(s_t) \left[\underbrace{v_t(s_t) - [r_{t+1} + \gamma v_t(s_{t+1})]}_{\text{TD target } \bar{v}_t} \right], \quad (3)$$

TD error δ_t

Here,

$$\bar{v}_t \doteq r_{t+1} + \gamma v_t(s_{t+1})$$

is called the **TD target**.

$$\delta_t \doteq v_t(s_t) - [r_{t+1} + \gamma v_t(s_{t+1})] = v_t(s_t) - \bar{v}_t$$

is called the **TD error**.

First, why is $\bar{v}_t$ called the TD target?

That is because the algorithm drives $v(s_t)$ towards $\bar{v}_t$.

To see that,

$$\begin{aligned} v_{t+1}(s_t) &= v_t(s_t) - \alpha_t(s_t) [v_t(s_t) - \bar{v}_t] \\ \implies v_{t+1}(s_t) - \bar{v}_t &= v_t(s_t) - \bar{v}_t - \alpha_t(s_t) [v_t(s_t) - \bar{v}_t] \\ \implies v_{t+1}(s_t) - \bar{v}_t &= [1 - \alpha_t(s_t)] [v_t(s_t) - \bar{v}_t] \\ \implies |v_{t+1}(s_t) - \bar{v}_t| &= [1 - \alpha_t(s_t)] |v_t(s_t) - \bar{v}_t| \end{aligned}$$

Since $\alpha_t(s_t)$ is a small positive number, we have

$$0 < 1 - \alpha_t(s_t) < 1$$

Therefore,

$$|v_{t+1}(s_t) - \bar{v}_t| \leq |v_t(s_t) - \bar{v}_t|$$

which means $v(s_t)$ is driven towards $\bar{v}_t$!

Second, what is the interpretation of the TD error?

$$\delta_t = v_t(s_t) - [r_{t+1} + \gamma v_t(s_{t+1})]$$

- It reflects the **difference** between **two time steps**.
- It reflects the **difference** between v_t and v_π . To see that, denote

$$\delta_{\pi,t} \doteq v_\pi(s_t) - [r_{t+1} + \gamma v_\pi(s_{t+1})]$$

Note that

$$\mathbb{E}[\delta_{\pi,t} | S_t = s_t] = v_\pi(s_t) - \mathbb{E}[R_{t+1} + \gamma v_\pi(s_{t+1}) | S_t = s_t] = 0.$$

- If $v_t = v_\pi$, then δ_t should be zero (in the expectation sense).
- Hence, if δ_t is not zero, then v_t is not equal to v_π .
- The TD error can be interpreted as **innovation**, which means new information obtained from the experience (s_t, r_{t+1}, s_{t+1}) .

- Connection between TD and RM(also 数学内涵)

$$v_{\pi}(s) = \mathbb{E}[R + \gamma v_{\pi}(S') | S = s], \quad s \in \mathcal{S}. \quad (5)$$

Equation (5) is another expression of the Bellman equation. It is sometimes called the **Bellman expectation equation**, an important tool to design and analyze TD algorithms.

In particular, by defining

$$g(v(s)) = v(s) - \mathbb{E}[R + \gamma v_{\pi}(S') | s],$$

we can rewrite (5) as

$$g(v(s)) = 0.$$

Since we can only obtain the samples r and s' of R and S' , the noisy observation we have is

$$\begin{aligned} \tilde{g}(v(s)) &= v(s) - [r + \gamma v_{\pi}(s')] \\ &= \underbrace{(v(s) - \mathbb{E}[R + \gamma v_{\pi}(S') | s])}_{g(v(s))} + \underbrace{(\mathbb{E}[R + \gamma v_{\pi}(S') | s] - [r + \gamma v_{\pi}(s')])}_{\eta}. \end{aligned}$$

Therefore, the RM algorithm for solving $g(v(s)) = 0$ is

$$\begin{aligned} v_{k+1}(s) &= v_k(s) - \alpha_k \tilde{g}(v_k(s)) \\ &= v_k(s) - \alpha_k (v_k(s) - [r_k + \gamma v_{\pi}(s'_k)]), \quad k = 1, 2, 3, \dots \end{aligned} \quad (6)$$

where $v_k(s)$ is the estimate of $v_{\pi}(s)$ at the k th step; r_k, s'_k are the samples of R, S' obtained at the k th step.

The **RM algorithm** in (6) looks very similar to the **TD algorithm**. However, there are **two differences**.

- Difference 1: The RM algorithm requires $\{(s, r_k, s'_k)\}$ for $k = 1, 2, 3, \dots$
 - Modification: $\{(s, r_k, s'_k)\}$ is changed to $\{(s_t, r_{t+1}, s_{t+1})\}$ so that the algorithm can utilize the sequential samples in an episode.
- Difference 2: The RM algorithm requires $v_{\pi}(s'_k)$.
 - Modification: $v_{\pi}(s'_k)$ is replaced by an estimate $v_t(s_{t+1})$.

With the above modifications, the RM algorithms becomes exactly the TD algorithm.

7.1.3 RL-Algorithm Classification: online/offline on-policy/off-policy

- Online vs Offline:
 - Offline: 模型policy的更新需要在agent与env交互完成生成整集trajectory episode后才能进行
 - Online: 模型policy的更新在agent与env交互的real-time实时进行: 每生成一组 $(s_t, a_t, r_{t+1}, s_{t+1})_t$ 即可更新
- On-policy vs Off-policy:
 - On-policy: behaviour policy 与 target policy 必须一致, 即需要采样的数据必须遵从target policy's distribution。为保证RL探索性, 需采取 ϵ -greedy 等具备探索性的更新
 - Off-policy: behaviour policy 与 target policy 不同。即update target policy 可以利用其他 policy 产生的数据。因而只要求behaviour policy 的探索性

7.2 TD Action Value Estimation (Sarsa) $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$

[similar to value estimation TD above, Sarsa is an action-value version of the value TD algorithm]

Suppose we have **some experience** $\{(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})\}_t$.

We can use the following *Sarsa* algorithm to estimate the action values:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - [r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})] \right],$$
$$q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t),$$

where $t = 0, 1, 2, \dots$

- $q_t(s_t, a_t)$ is an estimate of $q_\pi(s_t, a_t)$;
- $\alpha_t(s_t, a_t)$ is the learning rate depending on s_t, a_t .

What does the Sarsa algorithm do mathematically? The expression of Sarsa suggests that it is a stochastic approximation algorithm solving the following equation:

$$q_\pi(s, a) = \mathbb{E} [R + \gamma q_\pi(S', A') | s, a], \quad \forall s, a.$$

The ultimate goal of RL is to find optimal policies.

To do that, we can **combine Sarsa with a policy improvement step**.

The combined algorithm is also called Sarsa.

Pseudocode: Policy searching by Sarsa

For each episode, do

 Generate a_0 at s_0 following $\pi_0(s_0)$

 If s_t ($t = 0, 1, 2, \dots$) is not the target state, do

 Collect an experience sample $(r_{t+1}, s_{t+1}, a_{t+1})$ given (s_t, a_t) : generate r_{t+1}, s_{t+1} by interacting with the environment; generate a_{t+1} following $\pi_t(s_{t+1})$.

 Update q -value for (s_t, a_t) :

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - (r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})) \right]$$

 Update policy for s_t :

$$\pi_{t+1}(a|s_t) = 1 - \frac{\epsilon}{|\mathcal{A}(s_t)|} (|\mathcal{A}(s_t)| - 1) \text{ if } a = \arg \max_a q_{t+1}(s_t, a)$$
$$\pi_{t+1}(a|s_t) = \frac{\epsilon}{|\mathcal{A}(s_t)|} \text{ otherwise}$$

$s_t \leftarrow s_{t+1}, a_t \leftarrow a_{t+1}$

7.3 n-step Sarsa

- From one uniform opinion to see Monte Carlo learning, Sarsa and n-step Sarsa. and their advantages

The definition of action value is

$$q_{\pi}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a].$$

The discounted return G_t can be written in different forms as

$$\text{Sarsa} \leftarrow G_t^{(1)} = R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1}),$$

$$G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 q_{\pi}(S_{t+2}, A_{t+2}),$$

$\vdots$

$$n\text{-step Sarsa} \leftarrow G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^n q_{\pi}(S_{t+n}, A_{t+n}),$$

$\vdots$

$$\text{MC} \leftarrow G_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

It should be noted that $G_t = G_t^{(1)} = G_t^{(2)} = G_t^{(n)} = G_t^{(\infty)}$, where the superscripts merely indicate the different decomposition structures of G_t .

- Data: n -step Sarsa needs $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, \dots, r_{t+n}, s_{t+n}, a_{t+n})$.

- Since $(r_{t+n}, s_{t+n}, a_{t+n})$ has not been collected at time t , we are not able to implement n -step Sarsa at step t . We need to wait until time $t+n$ to update the q-value of (s_t, a_t) :

$$q_{t+n}(s_t, a_t) = q_{t+n-1}(s_t, a_t) - \alpha_{t+n-1}(s_t, a_t) [q_{t+n-1}(s_t, a_t) - (r_{t+1} + \gamma r_{t+2} + \dots + \gamma^n q_{t+n-1}(s_{t+n}, a_{t+n}))]$$

- Since n -step Sarsa includes Sarsa and MC learning as two extreme cases, its performance is a blend of Sarsa and MC learning:

- If n is large, its performance is close to MC learning and hence has a large variance but a small bias.
- If n is small, its performance is close to Sarsa and hence has a relatively large bias due to the initial guess and relatively low variance.

- Finally, n -step Sarsa is also for policy evaluation. It can be combined with the policy improvement step to search for optimal policies.

7.4 Q-learning (TD for optimal action values)

The Q-learning algorithm is

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) [q_t(s_t, a_t) - (r_{t+1} + \gamma \max_{a \in \mathcal{A}} q_t(s_{t+1}, a))],$$

$$q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t),$$

Q-learning is very similar to Sarsa. They are different only in terms of the TD target:

- The TD target in Q-learning is $r_{t+1} + \gamma \max_{a \in \mathcal{A}} q_t(s_{t+1}, a)$
- The TD target in Sarsa is $r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$

What does Q-learning do mathematically?

It aims to solve

$$q(s, a) = \mathbb{E} [R_{t+1} + \gamma \max_a q(S_{t+1}, a) | S_t = s, A_t = a], \quad \forall s, a.$$

This is the Bellman optimality equation expressed in terms of action values.

See the proof in my book.

As a result, Q-learning can directly estimate the optimal action values instead of action values of a given policy.

Pseudocode: Policy searching by Q-learning (on-policy version)

For each episode, do

If s_t ($t = 0, 1, 2, \dots$) is not the target state, do

Collect the experience sample (a_t, r_{t+1}, s_{t+1}) given s_t : generate a_t following $\pi_t(s_t)$; generate r_{t+1}, s_{t+1} by interacting with the environment.

Update q-value for (s_t, a_t) :

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) [q_t(s_t, a_t) - (r_{t+1} + \gamma \max_a q_t(s_{t+1}, a))]$$

Update policy for s_t :

$$\pi_{t+1}(a|s_t) = 1 - \frac{\epsilon}{|\mathcal{A}(s_t)|} (|\mathcal{A}(s_t)| - 1) \text{ if } a = \arg \max_a q_{t+1}(s_t, a)$$

$$\pi_{t+1}(a|s_t) = \frac{\epsilon}{|\mathcal{A}(s_t)|} \text{ otherwise}$$

Pseudocode: Optimal policy search by Q-learning (off-policy version)

Goal: Learn an optimal target policy π_T for all states from the experience samples generated by π_b .

For each episode $\{s_0, a_0, r_1, s_1, a_1, r_2, \dots\}$ generated by π_b , do

For each step $t = 0, 1, 2, \dots$ of the episode, do

Update q-value for (s_t, a_t) :

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) [q_t(s_t, a_t) - (r_{t+1} + \gamma \max_a q_t(s_{t+1}, a))]$$

Update target policy for s_t :

$$\pi_{T,t+1}(a|s_t) = 1 \text{ if } a = \arg \max_a q_{t+1}(s_t, a)$$

$$\pi_{T,t+1}(a|s_t) = 0 \text{ otherwise}$$

[这边update policy也可以在最后只进行一步update即可，前面的迭代都是在update q-value estimation，向最优的distribution偏向建模。这边是一个value-based方法，可以与后续的政策梯度的policy-based作比较]

7.5 A Unified Point of View

All the algorithms we introduced in this lecture can be expressed in a unified expression:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)[q_t(s_t, a_t) - \bar{q}_t]$$

where $\bar{q}_t$ is the TD target.

Different TD algorithms have different $\bar{q}_t$.

Algorithm	Expression of $\bar{q}_t$
Sarsa	$\bar{q}_t = r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$
n-step Sarsa	$\bar{q}_t = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^n q_t(s_{t+n}, a_{t+n})$
Q-learning	$\bar{q}_t = r_{t+1} + \gamma \max_a q_t(s_{t+1}, a)$
Monte Carlo	$\bar{q}_t = r_{t+1} + \gamma r_{t+2} + \dots$

Remark: The MC method can also be expressed in this unified expression by setting $\alpha_t(s_t, a_t) = 1$. In particular, the expression is $q_{t+1}(s_t, a_t) = \bar{q}_t$.

All the TD algorithms can be viewed as stochastic approximation algorithms solving the Bellman equation or Bellman optimality equation:

Algorithm	Equation to solve
Sarsa	BE: $q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) S_t = s, A_t = a]$
n-step Sarsa	BE: $q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \dots + \gamma^n q_\pi(s_{t+n}, a_{t+n}) S_t = s, A_t = a]$
Q-learning	BOE: $q(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_a q(S_{t+1}, a) S_t = s, A_t = a]$
Monte Carlo	BE: $q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \dots S_t = s, A_t = a]$

8、Value Function Methods

Code block

- 1 So far, state and action values are represented by tables.
- 2 • Advantage: intuitive and easy to analyze
- 3 • Disadvantage: difficult to handle large or continuous state or action spaces.
- 4 Two aspects: 1) storage; 2) generalization ability
- 5
- 6 from table to function
- 7 Advantage related to table:
- 8 • 1) storage: the number of function parameter is less than all values
- 9 • 2) generalization: visit state s, update value function will change filed around s

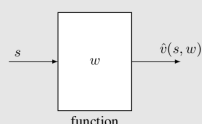
8.0 Summary

- From table record to function approximation. Storage and update change!
- DQN: two neural network + replay buffer

8.1 table vs function

Difference 1: How to retrieve the value of a state

- When the values are represented by a table, we can directly read the value in the table.
- When the values are represented by a function, we need to input the state index s into the function and calculate the function value.



For example, $s \rightarrow \phi(s) \rightarrow \phi^T(s)w = \hat{v}(s, w)$

Difference 2: How to update the value of a state



(a) Tabular method



(b) Function method

Benefit: generalization ability. When we update $\hat{v}(s, w)$ by changing w , the values of the neighboring states are also changed.

Remark:

1. $\phi(s)$ 为state的feature vector, ω 为function的参数 vector。上图for example的为linear model.
 - a. $|\phi(s)| \uparrow$ 即维数越高, 表示能力越强, value fit more accurately
 - b. 该模型难点在于 $\phi(s)$ 的选择
2. Generalization also brings cost: hardly to express one value accurately, just is called approximation
3. Idea approximate values using parameterized function: $\hat{v}(s; \omega) \rightarrow v_\pi(s)$
4. 下述依旧从state value为introduction再过渡到q-value结合policy update的脉络进行讲解

8.2 State Value Function Approximation

8.2.1 Problem Setting

- Let $v_\pi(s)$ and $\hat{v}(s, w)$ be the **true state value** and the **estimated state value**, respectively.
- Our goal is to find an **optimal w** so that $\hat{v}(s, w)$ can best approximate $v_\pi(s)$ for every s .
- This is a **policy evaluation problem**. Later we will extend to policy improvement.

To find the optimal w , we need **two steps**.

- The first step is to define an objective function.
- The second step is to derive algorithms for optimizing the objective function.

The **objective function** is

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2].$$

- Our **goal** is to find the best w that can minimize $J(w)$.
- The expectation is with respect to the random variable $S \in \mathcal{S}$.
What is the probability distribution of S ?
 - This is new. We have not discussed the probability distribution of states so far.
 - There are several ways to define the probability distribution of S .

8.2.2 Optimization Object Function for Distribution

The first way is to use a **uniform distribution**.

- That is to treat all the states to be **equally important** by setting the probability of each state as $1/|S|$.

- In this case, the objective function becomes

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2] = \frac{1}{|S|} \sum_{s \in S} (v_\pi(s) - \hat{v}(s, w))^2.$$

- **Drawback:**

- **The states may not be equally important.** For example, some states may be rarely visited by a policy. Hence, this way does not consider the real dynamics of the Markov process under the given policy.

The second way is to use the **stationary distribution**.

- Stationary distribution is an important concept that will be frequently used in this course. It describes the **long-run behavior** of a Markov process.
- Let $\{d_\pi(s)\}_{s \in S}$ denote the stationary distribution of the Markov process under policy π . By definition, $d_\pi(s) \geq 0$ and $\sum_{s \in S} d_\pi(s) = 1$.
- The objective function can be rewritten as

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2] = \sum_{s \in S} d_\pi(s) (v_\pi(s) - \hat{v}(s, w))^2.$$

This function is a weighted squared error.

- Since more frequently visited states have higher values of $d_\pi(s)$, their weights in the objective function are also higher than those rarely visited states.

Remark: More explanation about stationary distribution

- **Distribution:** Distribution of the state
- **Stationary:** Long-run behavior
- **Summary:** after the agent runs a long time following a policy, the probability that the agent is at any state can be described by this distribution.

Remarks:

- Stationary distribution is also called **steady-state distribution**, or **limiting distribution**.
- It is critical to understand the **value function method**.
- It is also important for the **policy gradient method** in the next lecture.

- Let $n_\pi(s)$ denote the number of times that s has been visited in a very long episode generated by π .
- Then, $d_\pi(s)$ can be approximated by

$$d_\pi(s) \approx \frac{n_\pi(s)}{\sum_{s' \in S} n_\pi(s')}$$

The converged values can be predicted because they are the entries of d_π :

$$d_\pi^T = d_\pi^T P_\pi$$

8.2.3 Optimization Algorithm

- To minimize the objective function $J(w)$, we can use the **gradient-descent** algorithm:

$$w_{k+1} = w_k - \alpha_k \nabla_w J(w_k)$$

The true gradient is

$$\begin{aligned} \nabla_w J(w) &= \nabla_w \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2] \\ &= \mathbb{E}[\nabla_w (v_\pi(S) - \hat{v}(S, w))^2] \\ &= 2\mathbb{E}[(v_\pi(S) - \hat{v}(S, w))(-\nabla_w \hat{v}(S, w))] \\ &= -2\mathbb{E}[(v_\pi(S) - \hat{v}(S, w))\nabla_w \hat{v}(S, w)] \end{aligned}$$

The true gradient above involves the calculation of an expectation.

- 再利用采样的stochastic gradient to replace the true expectation, 对于unavailable $v_\pi(s)$ 可以 MC or TD

$$w_{k+1} = w_k + \alpha_k \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))\nabla_w \hat{v}(S, w)]$$

$\Downarrow$

$$w_{t+1} = w_t + \alpha_t (v_\pi(s_t) - \hat{v}(s_t, w_t))\nabla_w \hat{v}(s_t, w_t)$$

where s_t is a sample of S . Here, $2\alpha_t$ is merged to α_t .

- The samples are expected to satisfy the stationary distribution. In practice, they may not satisfy.
- This algorithm is **not implementable** because it requires the true state value v_π , which is the unknown to be estimated.
- We can **replace** $v_\pi(s_t)$ with an approximation so that the algorithm is implementable.

- First, **Monte Carlo learning with function approximation**

Let g_t be the discounted return starting from s_t in the episode. Then, g_t can be used to approximate $v_\pi(s_t)$. The algorithm becomes

$$w_{t+1} = w_t + \alpha_t (g_t - \hat{v}(s_t, w_t))\nabla_w \hat{v}(s_t, w_t).$$

- Second, **TD learning with function approximation**

By the spirit of TD learning, $r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t)$ can be viewed as an approximation of $v_\pi(s_t)$. Then, the algorithm becomes

$$w_{t+1} = w_t + \alpha_t [r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)] \nabla_w \hat{v}(s_t, w_t).$$

8.2.4 Selection of Function Approximators

- Linear function: $\hat{v}(s; w) = \phi^T(s)w$; neural network as a nonlinear function, where parameter is w

In the linear case where $\hat{v}(s, w) = \phi^T(s)w$, we have

$$\nabla_w \hat{v}(s, w) = \phi(s).$$

Substituting the gradient into the TD algorithm

$$w_{t+1} = w_t + \alpha_t [r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)] \nabla_w \hat{v}(s_t, w_t)$$

yields

$$w_{t+1} = w_t + \alpha_t [r_{t+1} + \gamma \phi^T(s_{t+1})w_t - \phi^T(s_t)w_t] \phi(s_t),$$

which is the algorithm of TD learning with linear function approximation. It is called **TD-Linear** in our course.

- **Disadvantages** of linear function methods:

- Difficult to select appropriate feature vectors.

- **Advantages** of linear function methods:

- The **theoretical properties** of the TD algorithm in the linear case can be much better understood than in the nonlinear case.
- Linear function approximation is still powerful in the sense that the **tabular representation is a special case of linear function representation**.

Remark: table form = linear function, where feature vector is one-hot vector

8.2.5 Summary of TD learning with value function

1) This story started from the objective function:

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2]$$

The objective function suggests that it is a policy evaluation problem.

2) The gradient-descent algorithm is

$$w_{t+1} = w_t + \alpha_t (v_\pi(s_t) - \hat{v}(s_t, w_t)) \nabla_w \hat{v}(s_t, w_t)$$

3) The true value function, which is unknown, in the algorithm is **replaced by an approximation**, leading to the algorithm:

$$w_{t+1} = w_t + \alpha_t [r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)] \nabla_w \hat{v}(s_t, w_t)$$

Although this story is very helpful to understand the basic idea, it is **not mathematically rigorous**.

8.2.6 Theoretical Analysis

- 实际上，上述给出的 w update 的 object function 并非严格的 $J(w)$

Different objective functions:

- **Objective function 1: True value error**

$$J_E(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2] = \|\hat{v}(w) - v_\pi\|_D^2$$

- **Objective function 2: Bellman error**

$$J_{BE}(w) = \|\hat{v}(w) - (r_\pi + \gamma P_\pi \hat{v}(w))\|_D^2 \doteq \|\hat{v}(w) - T_\pi(\hat{v}(w))\|_D^2,$$

where $T_\pi(x) \doteq r_\pi + \gamma P_\pi x$

- **Objective function 3: Projected Bellman error**

$$J_{PBE}(w) = \|\hat{v}(w) - MT_\pi(\hat{v}(w))\|_D^2,$$

where M is a projection matrix.

- **The TD-Linear algorithm minimizes the projected Bellman error.**

More details are omitted here. Interested readers can check my book.

8.3 Sarsa with Function Approximation (q-value function approximation + policy update)

The Sarsa algorithm with value function approximation is

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t) \right] \nabla_w \hat{q}(s_t, a_t, w_t).$$

This is the same as the algorithm we introduced previously in this lecture except that $\hat{v}$ is replaced by $\hat{q}$.

Pseudocode: Sarsa with function approximation

Initialization: Initial parameter w_0 . Initial policy π_0 . $\alpha_t = \alpha > 0$ for all t . $\epsilon \in (0, 1)$.

Goal: Learn an optimal policy to lead the agent to the target state from an initial state s_0 .

For each episode, do

Generate a_0 at s_0 following $\pi_0(s_0)$

If s_t ($t = 0, 1, 2, \dots$) is not the target state, do

Collect the experience sample $(r_{t+1}, s_{t+1}, a_{t+1})$ given (s_t, a_t) : generate r_{t+1}, s_{t+1} by interacting with the environment; generate a_{t+1} following $\pi_t(s_{t+1})$.

Update q-value (update parameter):

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t) \right] \nabla_w \hat{q}(s_t, a_t, w_t)$$

Update policy:

$$\pi_{t+1}(a|s_t) = 1 - \frac{\epsilon}{|\mathcal{A}(s_t)|} (|\mathcal{A}(s_t)| - 1) \text{ if } a = \arg \max_{a \in \mathcal{A}(s_t)} \hat{q}(s_t, a, w_{t+1})$$

$$\pi_{t+1}(a|s_t) = \frac{\epsilon}{|\mathcal{A}(s_t)|} \text{ otherwise}$$

$$s_t \leftarrow s_{t+1}, a_t \leftarrow a_{t+1}$$

8.4 Q-learning with function approximation

- Similar to Sarsa, tabular Q-learning can also be extended to the case of value function approximation.

The q-value update rule is

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a, w_t) - \hat{q}(s_t, a_t, w_t) \right] \nabla_w \hat{q}(s_t, a_t, w_t),$$

which is the same as Sarsa except that $\hat{q}(s_{t+1}, a_{t+1}, w_t)$ is replaced by $\max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a, w_t)$.

Pseudocode: Q-learning with function approximation (on-policy version)

Initialization: Initial parameter w_0 . Initial policy π_0 . $\alpha_t = \alpha > 0$ for all t . $\epsilon \in (0, 1)$.

Goal: Learn an optimal path to lead the agent to the target state from an initial state s_0 .

For each episode, do

 If s_t ($t = 0, 1, 2, \dots$) is not the target state, do

 Collect the experience sample (a_t, r_{t+1}, s_{t+1}) given s_t : generate a_t following $\pi_t(s_t)$; generate r_{t+1}, s_{t+1} by interacting with the environment.

 Update value (update parameter):

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a, w_t) - \hat{q}(s_t, a_t, w_t) \right] \nabla_w \hat{q}(s_t, a_t, w_t)$$

 Update policy:

$$\begin{aligned} \pi_{t+1}(a|s_t) &= 1 - \frac{\epsilon}{|\mathcal{A}(s_t)|} (|\mathcal{A}(s_t)| - 1) \text{ if } a = \arg \max_{a \in \mathcal{A}(s_t)} \hat{q}(s_t, a, w_{t+1}) \\ \pi_{t+1}(a|s_t) &= \frac{\epsilon}{|\mathcal{A}(s_t)|} \text{ otherwise} \end{aligned}$$

8.5 Deep Q-learning (DQN)

8.5.1 Why dose the above w update parameter work poorly in neural network ?

1. 数据的独立同分布问题 (打破了 i.i.d 假设)

- 公式中的做法：这个公式是在线的 (online)、单步的 (step-by-step) 更新。智能体在环境中走一步，得到一个 $(s_t, a_t, r_{t+1}, s_{t+1})$ ，就立刻计算一次梯度并更新权重 w_t 。
- 神经网络的痛点：神经网络的梯度下降 (SGD及其变体) 有一个基本假设：训练数据是独立同分布 (i.i.d) 的。但强化学习中连续的时间步数据是高度相关的 (你前一秒在路口，下一秒一定在路口附近)。如果用高度相关的数据流去连续更新神经网络，网络会产生“灾难性遗忘” (忘了之前学过的状态)，导致参数震荡，陷入局部最优。
- DQN 的解法：引入经验回放池 (Experience Replay)。不再走一步更新一步，而是把大量的数据存到一个池子里，每次训练时从池子里随机抽取一个 Mini-batch 来计算梯度。这就打破了数据的相关性，符合了神经网络的训练方式。

2. “移动的靶子”导致震荡 (Moving Target Problem)

- 公式中的做法：注意看公式方括号里的部分，即时序差分目标 (TD Target) : $Y_t = r_{t+1} + \gamma \max_a \hat{q}(s_{t+1}, a, w_t)$ 。
- 神经网络的痛点：我们的目标是让当前的预测值 $\hat{q}(s_t, a_t, w_t)$ 逼近这个 Y_t 。但是，目标 Y_t 里面也包含了当前正在更新的参数 w_t ！这意味着，每次你通过梯度更新了 w_t 以靠近目标，目标本身也跟着移动了 (就像狗追自己的尾巴)。对于表达能力有限的线性函数，这种做法勉强还能收敛；但深度神经网络拟合能力极强，这种“目标跟随参数不断变化”的微小扰动会被迅速放大，导致 Q 值剧烈震荡甚至发散至无穷大。
- DQN 的解法：引入目标网络 (Target Network)。DQN 实际上使用了两套神经网络：一个是不断更新的当前网络 w ，另一个是作为目标的冻结网络 w^- 。TD Target 变成了 $r_{t+1} + \gamma \max_a \hat{q}(s_{t+1}, a, w^-)$ 。这样靶子就被固定住了，每隔几百步才把 w 复制给 w^- 移动一次靶子，使得神经网络的训练过程稳定下来。

Remark: 这两个DQN的解法也会是下文详述的Technique1/2

8.5.2 Back to Original Object Funtion $\mathcal{J}$

- That is because, in this objective function

$$J(w) = \mathbb{E} \left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w) \right)^2 \right],$$

the parameter w not only appears in $\hat{q}(S, A, w)$ but also in

$$y \doteq R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w)$$

- Since the optimal a depends on w ,

$$\nabla_w y \neq \gamma \max_{a \in \mathcal{A}(S')} \nabla_w \hat{q}(S', a, w)$$

- To solve this problem, we can assume that w in y is fixed (at least for a while) when we calculate the gradient.

Remark: Challenge is the gradient of y on w is hardly calculated. To solve it, introduce Technique 1: two network

8.5.3 Technique 1: Two Network

- One is a **main network** representing $\hat{q}(s, a, w)$
- The other is a **target network** $\hat{q}(s, a, w_T)$.

The objective function in this case degenerates to

$$J = \mathbb{E} \left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w_T) - \hat{q}(S, A, w) \right)^2 \right],$$

where w_T is the target network parameter.

When w_T is fixed, the gradient of J can be easily obtained as

$$\nabla_w J = \mathbb{E} \left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w_T) - \hat{q}(S, A, w) \right) \nabla_w \hat{q}(S, A, w) \right].$$

Implementation details:

- Let w and w_T denote the parameters of the main and target networks, respectively. They are set to be the same initially.
- In every iteration, we draw a mini-batch of samples $\{(s, a, r, s')\}$ from the replay buffer (will be explained later).
- For every (s, a, r, s') , we can calculate the desired output as

$$y_T \doteq r + \gamma \max_{a \in \mathcal{A}(s')} \hat{q}(s', a, w_T)$$

Therefore, we obtain a mini-batch of data:

$$\{(s, a, y_T)\}$$

- Use $\{(s, a, y_T)\}$ to train the network so as to minimize $(y_T - \hat{q}(s, a, w))^2$.

8.5.4 Technique 2: Experience Replay

- Why need experience replay?
 - Necessarity:
 - i. Neural network training data i.i.d.的要求, 放进replay buffer中break the correlation between consequent samples
 - ii. Object fuction J 中的expetation是对于 S 、 A 的, 因此是遵循policy's distribution进行 i.i.d采样的, yet not sample consequently or sample once
 - Advantage:

i. 提高样本数据利用效率

- Implementation

Question: What is experience replay?

Answer:

- After we have collected some experience samples, we do NOT use these samples **in the order they were collected**.
- Instead, we store them in a set, called **replay buffer** $\mathcal{B} \doteq \{(s, a, r, s')\}$
- Every time we train the neural network, we can **draw a mini-batch** of random samples from the replay buffer.
- The draw of samples, or called **experience replay**, should follow a **uniform distribution**.

Pseudocode: Deep Q-learning (off-policy version)

Initialization: A main network and a target network with the same initial parameter.

Goal: Learn an optimal target network to approximate the *optimal* action values from the experience samples generated by a given behavior policy π_b .

Store the experience samples generated by π_b in a replay buffer $\mathcal{B} = \{(s, a, r, s')\}$

For each iteration, do

Uniformly draw a mini-batch of samples from $\mathcal{B}$

For each sample (s, a, r, s') , calculate the target value as $y_T = r + \gamma \max_{a \in \mathcal{A}(s')} \hat{q}(s', a, w_T)$, where w_T is the parameter of the target network

Update the main network to minimize $(y_T - \hat{q}(s, a, w))^2$ using the mini-batch of samples

Set $w_T = w$ every C iterations

Remarks:

- Why no policy update?
- The network input and output are different from the DQN paper.

Remark:

- 因为只需要最后更新一次即可
- 原文采用neural network的是输入state，输出所有有限个action的q-value

9. Policy Gradient Methods

```

1 from value-based methods to policy-based methods
2 from value function methods to policy function methods(or called policy
  gradient methods)
3
4 value-based 即强体现在Q-learning中可以在迭代中update q-value function，在迭代后再
  update policy
5 而无需在迭代中一并update policy
6
7 why to policy function: (类似于value-function)
8 当action space A不再是离散稀疏的，而是离散稠密的or连续的。那么table form就不再可行。

```

- Now, policies can be represented by parameterized functions: $\pi(a|s; \theta)$, where $\theta \in \mathbb{R}^m$ is a parameter vector

9.0 Summary

The basic idea of the policy gradient is simple:

- First, metrics (or objective functions) to define optimal policies: $J(\theta)$, which can define optimal policies.
- Second, gradient-based optimization algorithms to search for optimal policies:

$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta} J(\theta_t)$$

Although the idea is simple, the complication emerges when we try to answer the following questions.

- What appropriate metrics should be used?
- How to calculate the gradients of the metrics?

These questions will be answered in detail in this lecture.

Contents of this lecture:

- Metrics for optimality
- Gradients of the metrics
- Gradient-ascent algorithm
- A special case: REINFORCE

Next lecture: Actor-critic

9.1 Difference Between function approximation and table record

- First, how to define optimal policies?

- In the tabular case, a policy π is optimal if it can maximize every state value.
- In the function case, a policy π is optimal if it can maximize certain scalar metrics.

- Second, how to access the probability of an action?

- In the tabular case, the probability of taking a at s can be directly accessed by looking up the tabular policy.
- In the function case, we need to calculate the value of $\pi(a|s, \theta)$ given the function structure and the parameter.

- Third, how to update policy? - table: change table value -function: change function parameter

9.2 Metrics to define optimal policies

- Remark:
 - policy-based体现在calculate metrics for optimal policy, instead of value for update policy (value-based). And the update policy by maximizing metrics. 换言之，因空间是非稀疏的，所以替换了discretized state的value-based的指标而可以重新定义metrics. 所以需要考虑how to define metrics

- 在之前的value-based的算法中，都是在某次迭代中对于state s_t ，估计其q-value，然后根据q-value update policy，但是update的时候只对table-policy π 中涉及到 s_t 的 $\pi(a|s_t)$ 进行update，对于其他state不作修改。但是走到policy function中，update policy的时候不再是对single state进行修改，而是修改function parameter θ ，会修改到多个state的表现(也就是generalization的原因)，因此不能靠单个single state q-value-based的指标来update，而应该考虑整体的metrics

9.2.1 Metric 1: average value

Case 1: d is independent of the policy π .

- This case is relatively simple because the gradient of the metric is easier to calculate: $\nabla_{\theta} \bar{v}_{\pi} = d^T \nabla_{\theta} v_{\pi}$
- In this case, we specifically denote d as d_0 and $\bar{v}_{\pi}$ as $\bar{v}_{\pi}^0$.

How to select d_0 ?

- One trivial way is to treat all the states equally important and hence select $d_0(s) = 1/|S|$.
- Another important case is that we are only interested in a specific state s_0 . For example, the episodes in some tasks always start from the same state s_0 . Then, we only care about the long-term return starting from s_0 . In this case,

$$d_0(s_0) = 1, \quad d_0(s \neq s_0) = 0$$

In this case, $\bar{v}_{\pi} = v_{\pi}(s_0)$

Case 2: d depends on the policy π .

- A common way is to select d as $d_{\pi}(s)$, which is the stationary distribution under π . Details of stationary distribution can be found in the last lecture and the book.
- The interpretation of selecting d_{π} is as follows.
 - d_{π} reflects the long-run behavior of the Markov decision process under a given policy π .
 - If one state is frequently visited in the long run, it is more important and deserves more weight.
 - If a state is hardly visited, then we give it less weight.

An important equivalent expression:

You will see the following metric often in the literature:

$$J(\theta) = \lim_{n \rightarrow \infty} \mathbb{E} \left[\sum_{t=0}^n \gamma^t R_{t+1} \right] = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \right].$$

Question: What is its relationship to the metric we introduced just now?

Answer: They are the same. That is because

$$\begin{aligned} J(\theta) &= \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \right] = \sum_{s \in S} d(s) \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} | S_0 = s \right] \\ &= \sum_{s \in S} d(s) v_{\pi}(s) \\ &= \bar{v}_{\pi} \end{aligned}$$

9.2.2 Metric 2: average reward

The second metric is average one-step reward or simply average reward:

$$\bar{r}_{\pi} \doteq \sum_{s \in S} d_{\pi}(s) r_{\pi}(s) = \mathbb{E}[r_{\pi}(S)],$$

where $S \sim d_{\pi}$,

$$r_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) r(s, a)$$

$$r(s, a) = \mathbb{E}[R|s, a] = \sum_r r p(r|s, a)$$

Remarks:

- $\bar{r}_{\pi}$ is simply a weighted average of immediate rewards.
- $r_{\pi}(s)$ is the average immediate reward that can be obtained from s .
- d_{π} is the stationary distribution.

An important fact is that

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{1}{n} \mathbb{E} \left[\sum_{t=0}^{n-1} R_{t+1} | S_0 = s_0 \right] &= \lim_{n \rightarrow \infty} \frac{1}{n} \mathbb{E} \left[\sum_{t=0}^{n-1} R_{t+1} \right] \\ &= \sum_s d_{\pi}(s) r_{\pi}(s) \\ &= \bar{r}_{\pi} \end{aligned}$$

Remarks:

- Highlight: the starting state s_0 does not matter.
- The derivation of the equation is nontrivial and can be found in my book.

9.2.3 Summary of the Two Metrics

Metric	Expression 1	Expression 2	Expression 3
$\bar{v}_\pi$	$\sum_{s \in \mathcal{S}} d(s) v_\pi(s)$	$\mathbb{E}_{S \sim d}[v_\pi(S)]$	$\lim_{n \rightarrow \infty} \mathbb{E}[\sum_{t=0}^n \gamma^t R_{t+1}]$
$\bar{r}_\pi$	$\sum_{s \in \mathcal{S}} d_\pi(s) r_\pi(s)$	$\mathbb{E}_{S \sim d_\pi}[r_\pi(S)]$	$\lim_{n \rightarrow \infty} \frac{1}{n} \mathbb{E}[\sum_{t=0}^{n-1} R_{t+1}]$

Table: Summary of the different but equivalent expressions of $\bar{v}_\pi$ and $\bar{r}_\pi$.

Remark 2 about the metrics:

- One complication is that the metrics can be defined in either the **discounted case** where $\gamma \in (0, 1)$ or the **undiscounted case** where $\gamma = 1$.
- The undiscounted case is nontrivial.
- We only consider the discounted case so far in this book. For details about the undiscounted case, see the book.

Remark 1 about the metrics:

- All these metrics are **functions of π** .
- Since π is parameterized by θ , these metrics are **functions of θ** .
- In other words, different values of θ can generate different metric values.

Therefore, we can search for the **optimal values of θ** to maximize these metrics. This is the basic idea of policy gradient methods.

Remark 3 about the metrics:

- What is the relationship between $\bar{r}_\pi$ and $\bar{v}_\pi$?
- The two metrics are **equivalent** (not equal) to each other. Specifically, in the discounted case where $\gamma < 1$, it holds that

$$\bar{r}_\pi = (1 - \gamma) \bar{v}_\pi.$$

Therefore, they can be **maximized simultaneously**. See the proof in the book.

9.3 Gradients of the metrics

$$\nabla_\theta J(\theta) = \sum_{s \in \mathcal{S}} \eta(s) \sum_{a \in \mathcal{A}} \nabla_\theta \pi(a|s, \theta) q_\pi(s, a)$$

The above is a **unified expression of many cases**:

- $J(\theta)$ can be $\bar{v}_\pi$, $\bar{r}_\pi$, or $\bar{v}_\pi^0$.
- “=” may denote strict equality, approximation, or proportional to.
- η is a distribution or weight of the states.

The derivation of this expression is **very complex**.

Details are not given here. Interested readers can read my book.

For most readers, it is sufficient to know this expression.

A compact and important expression of the gradient:

$$\begin{aligned} \nabla_\theta J(\theta) &= \sum_{s \in \mathcal{S}} \eta(s) \sum_{a \in \mathcal{A}} \nabla_\theta \pi(a|s, \theta) q_\pi(s, a) \\ &= \mathbb{E}_{S \sim \eta, A \sim \pi} [\nabla_\theta \ln \pi(A|S, \theta) q_\pi(S, A)] \end{aligned}$$

First, why is this expression useful?

- Because we can use samples to approximate the gradient:

$$\nabla_\theta J \approx \nabla_\theta \ln \pi(a|s, \theta) q_\pi(s, a)$$

where s, a are samples. This is the idea of stochastic gradient descent.

Remark: $\pi(a|s; \theta) \in (0, 1)$ 因为在神经网络中after last dense layer softmax normalization, 所以 π is stachastic and hence exploratory

9.4 Gradient-ascent Algorithm

1) The gradient-ascent algorithm maximizing $J(\theta)$ is

$$\begin{aligned} \theta_{t+1} &= \theta_t + \alpha \nabla_\theta J(\theta_t) \\ &= \theta_t + \alpha \mathbb{E} [\nabla_\theta \ln \pi(A|S, \theta_t) q_\pi(S, A)] \end{aligned}$$

2) Since the true gradient is unknown, we can replace it by a stochastic one:

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t, \theta_t) q_\pi(s_t, a_t)$$

3) Furthermore, since q_π is unknown, it can be replaced by an estimate:

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t, \theta_t) q_t(s_t, a_t)$$

- If $q_\pi(s_t, a_t)$ is estimated by Monte Carlo estimation, the algorithm has a specific name, **REINFORCE**.
- REINFORCE is one of earliest and simplest policy gradient algorithms.
- Many other policy gradient algorithms such as the actor-critic methods can be obtained by extending REINFORCE (next lecture).

Pseudocode: Policy Gradient by Monte Carlo (REINFORCE)

Initialization: Initial parameter θ ; $\gamma \in (0, 1)$; $\alpha > 0$.

Goal: Learn an optimal policy to maximize $J(\theta)$.

For each episode, do

Generate an episode $\{s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T\}$ following $\pi(\theta)$.

For $t = 0, 1, \dots, T - 1$:

Value update: $q_t(s_t, a_t) = \sum_{k=t+1}^T \gamma^{k-t-1} r_k$

Policy update: $\theta \leftarrow \theta + \alpha \nabla_\theta \ln \pi(a_t|s_t, \theta) q_t(s_t, a_t)$

Remark 1: How to do sampling?

$$\mathbb{E}_{S \sim \eta, A \sim \pi} \left[\nabla_{\theta} \ln \pi(A|S, \theta_t) q_{\pi}(S, A) \right] \longrightarrow \nabla_{\theta} \ln \pi(a|s, \theta_t) q_{\pi}(s, a)$$

- How to sample S ?
 - $S \sim \eta$, where the distribution η is a long-run behavior under π .
 - In practice, people usually do not care about it.
- How to sample A ?
 - $A \sim \pi(A|S, \theta)$. Hence, a_t should be sampled following $\pi(\theta_t)$ at s_t .
 - Therefore, **policy gradient methods are on-policy**.

Remark: REINFORCE是on-policy and offline, 因而效率特别低, each finished trajectory, each train episode, ...

Remark 2: How to interpret this algorithm?

Since

$$\nabla_{\theta} \ln \pi(a_t|s_t, \theta_t) = \frac{\nabla_{\theta} \pi(a_t|s_t, \theta_t)}{\pi(a_t|s_t, \theta_t)}$$

the algorithm can be rewritten as

$$\begin{aligned} \theta_{t+1} &= \theta_t + \alpha \nabla_{\theta} \ln \pi(a_t|s_t, \theta_t) q_t(s_t, a_t) \\ &= \theta_t + \alpha \underbrace{\left(\frac{q_t(s_t, a_t)}{\pi(a_t|s_t, \theta_t)} \right)}_{\beta_t} \nabla_{\theta} \pi(a_t|s_t, \theta_t). \end{aligned}$$

Therefore, we have the important expression of the algorithm:

$$\theta_{t+1} = \theta_t + \alpha \beta_t \nabla_{\theta} \pi(a_t|s_t, \theta_t)$$

$$\theta_{t+1} = \theta_t + \alpha \underbrace{\left(\frac{q_t(s_t, a_t)}{\pi(a_t|s_t, \theta_t)} \right)}_{\beta_t} \nabla_{\theta} \pi(a_t|s_t, \theta_t)$$

Interpretation (continued): β_t can **balance exploration and exploitation**.

The reason is as follows.

- First, β_t is **proportional to** $q_t(s_t, a_t)$.

$$\text{greater } q_t(s_t, a_t) \implies \text{greater } \beta_t \implies \text{greater } \pi(a_t|s_t, \theta_{t+1})$$

Therefore, the algorithm intends to **exploit** actions with greater values.

- Second, β_t is **inversely proportional to** $\pi(a_t|s_t, \theta_t)$ (when $q_t(s_t, a_t) > 0$).

$$\text{smaller } \pi(a_t|s_t, \theta_t) \implies \text{greater } \beta_t \implies \text{greater } \pi(a_t|s_t, \theta_{t+1})$$

Therefore, the algorithm intends to **explore** actions that have low probabilities.

10. Actor-Critic Methods

Code block

- 1 从REINFORCE中estimate q-value的方法是Monte Carlo。
- 2 对应地, 可以采用TD来estimate q-value,
- 3 并且, state space和action space通常是相同的或‘相对应的’, 在policy gradient中action space稠密的
- 4 那么state space也是稠密的, 因而会采用value **function** methods
- 5 所以Actor-Critic Methods:
- 6 Critic: Value Function(TD style)
- 7 Actor: Policy Function

10.0 Summary

- In the previous chapter: REINFORCE = Monte Carlo Estimation + policy gradient
- QAC = TD-style value functionpolicy + policy gradient
- A2C = replace QAC q-value with advantage to minimize sample variance
- Off-policy actor-critic: importance sample change action distribution from target policy to behaviour policy
- DPG: directly generate action a , action probability is unavailable. Change object function(metrics): state distribution independent to policy, without action distribution. It hence becomes off-policy

10.1 The simplest actor-critic (QAC)

Revisit the idea of policy gradient introduced in the last lecture.

- 1) A scalar metric $J(\theta)$, which can be $\bar{v}_\pi$ or $\bar{r}_\pi$.
- 2) The gradient-ascent algorithm maximizing $J(\theta)$ is

$$\begin{aligned}\theta_{t+1} &= \theta_t + \alpha \nabla_\theta J(\theta_t) \\ &= \theta_t + \alpha \mathbb{E}_{S \sim \eta, A \sim \pi} \left[\nabla_\theta \ln \pi(A|S, \theta_t) q_\pi(S, A) \right]\end{aligned}$$

- 3) The stochastic gradient-ascent algorithm is

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t, \theta_t) q_t(s_t, a_t)$$

This expression is very important! We can directly see "actor" and "critic" from it:

- This expression corresponds to actor!
- The algorithm estimating $q_t(s, a)$ corresponds to critic!

How to get $q_t(s_t, a_t)$?

So far, we have studied **two ways** to estimate action values:

- **Monte Carlo learning:** If MC is used, the corresponding algorithm is called **REINFORCE** or **Monte Carlo policy gradient**.
 - We introduced in the last lecture.
- **Temporal-difference learning:** If TD is used, such kind of algorithms are usually called **actor-critic**.
 - We will introduce in this lecture.

The simplest actor-critic algorithm (QAC)

Initialization: A policy function $\pi(a|s, \theta_0)$ where θ_0 is the initial parameter. A value function $q(s, a, w_0)$ where w_0 is the initial parameter. $\alpha_w, \alpha_\theta > 0$.
Goal: Learn an optimal policy to maximize $J(\theta)$.

At time step t in each episode, do

Generate a_t following $\pi(a|s_t, \theta_t)$, observe r_{t+1}, s_{t+1} , and then generate a_{t+1} following $\pi(a|s_{t+1}, \theta_t)$.

Actor (policy update):

$$\theta_{t+1} = \theta_t + \alpha_\theta \nabla_\theta \ln \pi(a_t|s_t, \theta_t) q(s_t, a_t, w_t)$$

Critic (value update):

$$w_{t+1} = w_t + \alpha_w [r_{t+1} + \gamma q(s_{t+1}, a_{t+1}, w_t) - q(s_t, a_t, w_t)] \nabla_w q(s_t, a_t, w_t)$$

Remarks:

- The *critic* corresponds to "SARSA+value function approximation".
- The *actor* corresponds to the policy update algorithm.
- This particular actor-critic algorithm is sometimes referred to as **Q Actor-Critic (QAC)**.
- Though simple, this algorithm reveals the core idea of actor-critic methods. It can be extended to generate many other algorithms as shown later.

10.2 Advantage actor-critic (A2C)

- Remark: 因为gradient的expectation变成stochastic gradient sample, 为了训练稳定收敛, 希望其variance littler

10.2.1 Baseline invariance

- Core idea: Introduce a baseline to reduce variance

Property: the policy gradient is invariant to an additional baseline:

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \mathbb{E}_{S \sim \eta, A \sim \pi} \left[\nabla_{\theta} \ln \pi(A|S, \theta_t) q_{\pi}(S, A) \right] \\ &= \mathbb{E}_{S \sim \eta, A \sim \pi} \left[\nabla_{\theta} \ln \pi(A|S, \theta_t) (q_{\pi}(S, A) - b(S)) \right]\end{aligned}$$

Here, the additional baseline $b(S)$ is a scalar function of S .

- Analysis valid and useful

First, why is it valid?

That is because

$$\mathbb{E}_{S \sim \eta, A \sim \pi} \left[\nabla_{\theta} \ln \pi(A|S, \theta_t) b(S) \right] = 0$$

The details:

$$\begin{aligned}\mathbb{E}_{S \sim \eta, A \sim \pi} \left[\nabla_{\theta} \ln \pi(A|S, \theta_t) b(S) \right] &= \sum_{s \in \mathcal{S}} \eta(s) \sum_{a \in \mathcal{A}} \pi(a|s, \theta_t) \nabla_{\theta} \ln \pi(a|s, \theta_t) b(s) \\ &= \sum_{s \in \mathcal{S}} \eta(s) \sum_{a \in \mathcal{A}} \nabla_{\theta} \pi(a|s, \theta_t) b(s) \\ &= \sum_{s \in \mathcal{S}} \eta(s) b(s) \sum_{a \in \mathcal{A}} \nabla_{\theta} \pi(a|s, \theta_t) \\ &= \sum_{s \in \mathcal{S}} \eta(s) b(s) \nabla_{\theta} \sum_{a \in \mathcal{A}} \pi(a|s, \theta_t) \\ &= \sum_{s \in \mathcal{S}} \eta(s) b(s) \nabla_{\theta} 1 = 0\end{aligned}$$

Second, why is the baseline useful?

The gradient is $\nabla_{\theta} J(\theta) = \mathbb{E}[X]$ where

$$X(S, A) \doteq \nabla_{\theta} \ln \pi(A|S, \theta_t) [q_{\pi}(S, A) - b(S)]$$

We have

- $\mathbb{E}[X]$ is invariant to $b(S)$.
- $\text{var}(X)$ is NOT invariant to $b(S)$.
 - Why? Because $\text{tr}[\text{var}(X)] = \mathbb{E}[X^T X] - \bar{x}^T \bar{x}$ and

$$\begin{aligned}\mathbb{E}[X^T X] &= \mathbb{E} \left[(\nabla_{\theta} \ln \pi)^T (\nabla_{\theta} \ln \pi) (q_{\pi}(S, A) - b(S))^2 \right] \\ &= \mathbb{E} \left[\|\nabla_{\theta} \ln \pi\|^2 (q_{\pi}(S, A) - b(S))^2 \right]\end{aligned}$$

- Choose one optimal baseline that can minimize $\text{Var}(X)$, for any s and also related to s

- The optimal baseline that can minimize $\text{var}(X)$ is, for any $s \in \mathcal{S}$,

$$b^*(s) = \frac{\mathbb{E}_{A \sim \pi} [\|\nabla_{\theta} \ln \pi(A|s, \theta_t)\|^2 q_{\pi}(s, A)]}{\mathbb{E}_{A \sim \pi} [\|\nabla_{\theta} \ln \pi(A|s, \theta_t)\|^2]}.$$

See the proof in my book.

- Although this baseline is optimal, it is complex.
- We can remove the weight $\|\nabla_{\theta} \ln \pi(A|s, \theta_t)\|^2$ and select the suboptimal baseline:

$$b(s) = \mathbb{E}_{A \sim \pi} [q_{\pi}(s, A)] = v_{\pi}(s)$$

which is the state value of s !

10.2.2 Algorithm of A2C

When $b(s) = v_\pi(s)$:

- The gradient-ascent algorithm is

$$\begin{aligned}\theta_{t+1} &= \theta_t + \alpha \mathbb{E} \left[\nabla_\theta \ln \pi(A|S, \theta_t) [q_\pi(S, A) - v_\pi(S)] \right] \\ &\doteq \theta_t + \alpha \mathbb{E} \left[\nabla_\theta \ln \pi(A|S, \theta_t) \delta_\pi(S, A) \right]\end{aligned}$$

where

$$\delta_\pi(S, A) \doteq q_\pi(S, A) - v_\pi(S)$$

is called the **advantage function** (why called advantage?).

- The stochastic version is

$$\begin{aligned}\theta_{t+1} &= \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t, \theta_t) [q_t(s_t, a_t) - v_t(s_t)] \\ &= \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t, \theta_t) \delta_t(s_t, a_t)\end{aligned}$$

Furthermore, the advantage function is approximated by the TD error:

$$\delta_t = q_t(s_t, a_t) - v_t(s_t) \rightarrow r_{t+1} + \gamma v_t(s_{t+1}) - v_t(s_t)$$

- This approximation is reasonable because

$$\mathbb{E}[q_\pi(S, A) - v_\pi(S) | S = s_t, A = a_t] = \mathbb{E}[R + \gamma v_\pi(S') - v_\pi(S) | S = s_t, A = a_t]$$

- Benefit:** only need one network to approximate $v_\pi(s)$ rather than two networks for $q_\pi(s, a)$ and $v_\pi(s)$.

Interpretation and implementation of A2C

Interpretation of the A2C algorithm:

$$\begin{aligned}\theta_{t+1} &= \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t, \theta_t) \delta_t(s_t, a_t) \\ &= \theta_t + \alpha \frac{\nabla_\theta \pi(a_t|s_t, \theta_t)}{\pi(a_t|s_t, \theta_t)} \delta_t(s_t, a_t) \\ &= \theta_t + \alpha \underbrace{\left(\frac{\delta_t(s_t, a_t)}{\pi(a_t|s_t, \theta_t)} \right)}_{\beta_t} \nabla_\theta \pi(a_t|s_t, \theta_t)\end{aligned}$$

Then,

$$\text{greater } \delta_t(s_t, a_t) \implies \text{greater } \beta_t \implies \text{greater } \pi(a_t|s_t, \theta_{t+1})$$

$$\text{smaller } \pi(a_t|s_t, \theta_t) \implies \text{greater } \beta_t \implies \text{greater } \pi(a_t|s_t, \theta_{t+1})$$

See the analysis of a similar case in the last lecture.

- It can **well balance exploration and exploitation**.
- What matters is the **relative value** δ_t rather than the **absolute value** q_t , which is more reasonable.

Advantage actor-critic (A2C) or TD actor-critic

Initialization: A policy function $\pi(a|s, \theta_0)$ where θ_0 is the initial parameter. A value function $v(s, w_0)$ where w_0 is the initial parameter. $\alpha_w, \alpha_\theta > 0$.

Goal: Learn an optimal policy to maximize $J(\theta)$.

At time step t in each episode, do

Generate a_t following $\pi(a|s_t, \theta_t)$ and then observe r_{t+1}, s_{t+1} .

Advantage (TD error):

$$\delta_t = r_{t+1} + \gamma v(s_{t+1}, w_t) - v(s_t, w_t)$$

Actor (policy update):

$$\theta_{t+1} = \theta_t + \alpha_\theta \delta_t \nabla_\theta \ln \pi(a_t|s_t, \theta_t)$$

Critic (value update):

$$w_{t+1} = w_t + \alpha_w \delta_t \nabla_w v(s_t, w_t)$$

10.3 off-policy actor-critic

- Policy gradient is in-policy due to

$$\nabla_\theta J(\theta) = \mathbb{E}_{S \sim \eta, A \sim \pi}[*]$$

10.3.1 Importance sample

Note that

$$\mathbb{E}_{X \sim p_0}[X] = \sum_x p_0(x)x = \sum_x p_1(x) \underbrace{\frac{p_0(x)}{p_1(x)}}_{f(x)} x = \mathbb{E}_{X \sim p_1}[f(X)]$$

- Thus, we can estimate $\mathbb{E}_{X \sim p_0}[X]$ by estimating $\mathbb{E}_{X \sim p_1}[f(X)]$.
- How to estimate $\mathbb{E}_{X \sim p_1}[f(X)]$? Easy. Let

$$\bar{f} \doteq \frac{1}{n} \sum_{i=1}^n f(x_i), \quad \text{where } x_i \sim p_1$$

Then,

$$\bar{f} \rightarrow \mathbb{E}_{X \sim p_1}[f(X)], \quad \text{as } n \rightarrow \infty$$

Therefore, $\bar{f}$ is a good approximation for $\mathbb{E}_{X \sim p_1}[f(X)] = \mathbb{E}_{X \sim p_0}[X]$

$$\mathbb{E}_{X \sim p_0}[X] \approx \bar{f} = \frac{1}{n} \sum_{i=1}^n f(x_i) = \frac{1}{n} \sum_{i=1}^n \frac{p_0(x_i)}{p_1(x_i)} x_i$$

- $\frac{p_0(x_i)}{p_1(x_i)}$ is called the **importance weight**.
 - If $p_1(x_i) = p_0(x_i)$, the importance weight is one and $\bar{f}$ becomes $\bar{x}$.
 - If $p_0(x_i) \geq p_1(x_i)$, x_i can be more often sampled by p_0 than p_1 . The importance weight (> 1) can emphasize the importance of this sample.

- Remark: 可能会有疑问如下:

You may ask: While $\bar{f} = \frac{1}{n} \sum_{i=1}^n \frac{p_0(x_i)}{p_1(x_i)} x_i$ requires $p_0(x)$, if I know $p_0(x)$, why not directly calculate the expectation?

Answer: We may only be able to obtain $p_0(x)$ of a given x , but not all x .

- For example, continuous case, complex expression of p_0 , or no expression of p_0 (e.g., p_0 represented by a neural network).

回到policy gradient中REINFORCE，为什么一定要把 $\pi(a|s; \theta)$ 提出来一起作为expectation's distribution并stochastic sample? 因为如果将 π 放在里边计算，但是q-value是Monte Carlo方法计算的，并没有每个 a 的trajectory计算不了Return

10.3.2 Theory and Algorithm

Theorem (Off-policy policy gradient theorem)

In the discounted case where $\gamma \in (0, 1)$, the gradient of $J(\theta)$ is

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \mathbb{E}_{S \sim \rho, A \sim \pi} [\nabla_{\theta} \ln \pi(A|S, \theta) q_{\pi}(S, A)] \\ &= \mathbb{E}_{S \sim \rho, A \sim \beta} \left[\frac{\pi(A|S, \theta)}{\beta(A|S)} \nabla_{\theta} \ln \pi(A|S, \theta) q_{\pi}(S, A) \right]\end{aligned}$$

where β is the behavior policy and ρ is a state distribution.

- 将q-value替换为advantage, remark: 因为advantage是用 $= r_{t+1} + \gamma v_w(s_{t+1}) - v_w(s_t)$ 计算得到的与 a 、 θ 无关,提出到梯度算子外

Off-policy actor-critic based on importance sampling

Initialization: A given behavior policy $\beta(a|s)$. A target policy $\pi(a|s, \theta_0)$ where θ_0 is the initial parameter. A value function $v(s, w_0)$ where w_0 is the initial parameter. $\alpha_w, \alpha_\theta > 0$.

Goal: Learn an optimal policy to maximize $J(\theta)$.

At time step t in each episode, do

Generate a_t following $\beta(s_t)$ and then observe r_{t+1}, s_{t+1} .

Advantage (TD error):

$$\delta_t = r_{t+1} + \gamma v(s_{t+1}, w_t) - v(s_t, w_t)$$

Actor (policy update):

$$\theta_{t+1} = \theta_t + \alpha_\theta \frac{\pi(a_t|s_t, \theta_t)}{\beta(a_t|s_t)} \delta_t \nabla_\theta \ln \pi(a_t|s_t, \theta_t)$$

Critic (value update):

$$w_{t+1} = w_t + \alpha_w \frac{\pi(a_t|s_t, \theta_t)}{\beta(a_t|s_t)} \delta_t \nabla_w v(s_t, w_t)$$

Remark: 为什么依旧要把 $\pi(a|s; \theta)$ 放到 expectation 中最 stochastic sample? 依旧是 q-value 的问题, 如果通过 $= r_{t+1} + \gamma v_w(s_{t+1}) - v_w(s_t)$ 计算的话需要 r_{t+1} data, 在有限数据中显然是不具备所有 action 对应 r_{t+1} 的。如果通过 $q_t(s, a)$ 那么这个 neural network 就与 a 有关, 其关于 θ 梯度需反向传播给 π , 这样使得梯度计算更复杂。并且若采用 advantage, 则需要两个网络一个估计 q-value, 一个估计 state-value

10.4 Deterministic actor-critic (DPG)

- 上文所讲的 policy neural network π 都更像是一个分类模型去输出有限离散 action 的概率, 此时转换向一个 deterministic policy 更像是一个生成模型 $a = \mu(s; \theta)$, one mapping from state to action directly.
- 此时 policy gradient 没办法使用, 因为根本无法得到生成 action 的概率

Consider the metric of average state value in the discounted case:

$$J(\theta) = \mathbb{E}[v_\mu(s)] = \sum_{s \in \mathcal{S}} d_0(s) v_\mu(s)$$

where $d_0(s)$ is a probability distribution satisfying $\sum_{s \in \mathcal{S}} d_0(s) = 1$.

- d_0 is selected to be independent of μ . The gradient in this case is easier to calculate.
- There are two special yet important cases of selecting d_0 .
 - The first special case is that $d_0(s_0) = 1$ and $d_0(s \neq s_0) = 0$, where s_0 is a specific starting state of interest.
 - The second special case is that d_0 is the stationary distribution of a behavior policy that is different from the μ .

Theorem (Deterministic policy gradient theorem in the discounted case)

In the discounted case where $\gamma \in (0, 1)$, the gradient of $J(\theta)$ is

$$\begin{aligned} \nabla_\theta J(\theta) &= \sum_{s \in \mathcal{S}} \rho_\mu(s) \nabla_\theta \mu(s) (\nabla_a q_\mu(s, a))|_{a=\mu(s)} \\ &= \mathbb{E}_{S \sim \rho_\mu} [\nabla_\theta \mu(S) (\nabla_a q_\mu(S, a))|_{a=\mu(S)}] \end{aligned}$$

Here, ρ_μ is a state distribution.

See more details and the proof in my book.

One important difference from the stochastic case:

- The gradient does not involve the distribution of the action A (why?).
- As a result, the deterministic policy gradient method is off-policy.

Remark:

- 因为无法得到 action 概率, 因此 metrics 选择的 state distribution independent with policy
- 这也是为什么 gradient doesn't involve the action distribution A

• 将expectation转换为stochastic sample得到algorithm implementation

Deterministic policy gradient or deterministic actor-critic	Remarks:
Initialization: A given behavior policy $\beta(a s)$. A deterministic target policy $\mu(s, \theta_0)$ where θ_0 is the initial parameter. A value function $q(s, a, w_0)$ where w_0 is the initial parameter. $\alpha_w, \alpha_\theta > 0$. Goal: Learn an optimal policy to maximize $J(\theta)$. At time step t in each episode, do Generate a_t following β and then observe r_{t+1}, s_{t+1}. TD error: $\delta_t = r_{t+1} + \gamma q(s_{t+1}, \mu(s_{t+1}, \theta_t), w_t) - q(s_t, a_t, w_t)$ Actor (policy update): $\theta_{t+1} = \theta_t + \alpha_\theta \nabla_\theta \mu(s_t, \theta_t) (\nabla_a q(s_t, a, w_t)) _{a=\mu(s_t)}$ Critic (value update): $w_{t+1} = w_t + \alpha_w \delta_t \nabla_w q(s_t, a_t, w_t)$	• This is an off-policy implementation where the behavior policy β may be different from μ. • β can also be replaced by $\mu + \text{noise}$. • How to select the function to represent $q(s, a, w)$? • Linear function: $q(s, a, w) = \phi^T(s, a)w$ where $\phi(s, a)$ is the feature vector. Details can be found in the DPG paper. • Neural networks: deep deterministic policy gradient (DDPG) method.